



Universidad Europea

UNIVERSIDAD EUROPEA DE MADRID

ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO

GRADO EN INGENIERÍA INFORMÁTICA

PROYECTO FIN DE GRADO

Diseño e implementación de una plataforma
corporativa de ciberinteligencia basada en OpenCTI
para el soporte de un servicio de Cyber Threat
Intelligence (CTI)

Manuel Araújo Baño

Dirigido por

Ignacio Cea Fornies

CURSO 2025-2026

Manuel Araújo Baño — Diseño e implementación de una plataforma corporativa de ciberinteligencia basada en OpenCTI para el soporte de un servicio de Cyber Threat Intelligence (CTI)

TÍTULO: Diseño e implementación de una plataforma corporativa de ciberinteligencia basada en OpenCTI para el soporte de un servicio de Cyber Threat Intelligence (CTI)

AUTOR: Manuel Araújo Baño

TITULACIÓN: GRADO EN INGENIERÍA INFORMÁTICA

DIRECTOR/ES DEL PROYECTO: Ignacio Cea Fornies

FECHA: mayo de 2026

RESUMEN

Las organizaciones se enfrentan a un volumen creciente de alertas de seguridad sin contexto suficiente para diferenciar amenazas reales de falsos positivos, fenómeno conocido como fatiga de alertas. A esto se suma la dependencia de soluciones comerciales de Cyber Threat Intelligence (CTI) costosas o de proveedores extraterritoriales que comprometen la soberanía del dato.

Este Trabajo de Fin de Grado, desarrollado de forma autónoma sin colaboración empresarial, aborda esta problemática mediante el diseño, implementación y validación de una plataforma corporativa de ciberinteligencia on-premise basada exclusivamente en software libre. La arquitectura integra cinco componentes operando de forma coordinada sobre un entorno virtualizado bajo control directo: Proxmox VE como hipervisor, OpenCTI como núcleo de inteligencia, Wazuh como SIEM/XDR (respuesta y detección extendida), OPNsense como firewall perimetral y n8n como orquestador SOAR.

Los principales resultados del proyecto son el desarrollo de conectores propios de ingesta y enriquecimiento para OpenCTI, y la implementación de dos flujos de orquestación complementarios: uno automatizado que reacciona a alertas críticas, y otro conversacional que permite al analista ejecutar acciones sobre la infraestructura mediante lenguaje natural a través de un agente IA con tool calling.

Como conclusiones, el trabajo confirma la viabilidad técnica y operativa de construir un servicio CTI corporativo íntegramente con software libre, sin renunciar a las capacidades de las soluciones comerciales y respetando los principios de soberanía del dato y cumplimiento normativo europeo. La arquitectura modular adoptada garantiza además la extensibilidad del sistema hacia futuras capacidades operativas.

Palabras clave: Cyber Threat Intelligence, OpenCTI, SOAR, soberanía del dato, agente IA, ciberseguridad

ABSTRACT

Organizations face a growing volume of security alerts without sufficient context to distinguish real threats from false positives, a phenomenon known as alert fatigue. This is compounded by reliance on costly commercial Cyber Threat Intelligence (CTI) solutions or extraterritorial providers that compromise data sovereignty.

This Bachelor's Thesis, developed autonomously without corporate collaboration, addresses this problem through the design, implementation and validation of a corporate on-premise cyber threat intelligence platform built exclusively with open-source software. The architecture integrates five components operating in a coordinated manner on a virtualized environment under direct control: Proxmox VE as hypervisor, OpenCTI as the intelligence core, Wazuh as SIEM/XDR, OPNsense as the perimeter firewall and n8n as the SOAR.

The main results of the project are the development of custom ingestion and enrichment connectors for OpenCTI, and the implementation of two complementary orchestration workflows: an automated one that reacts to critical alerts, and a conversational one that allows the analyst to execute actions on the infrastructure through natural language via an AI agent with tool calling capabilities.

In conclusion, the work confirms the technical and operational viability of building a corporate CTI service entirely with open-source software, without giving up the capabilities of commercial solutions and respecting the principles of data sovereignty and European regulatory compliance. The modular architecture adopted also ensures the extensibility of the system towards future operational capabilities.

Keywords: Cyber Threat Intelligence, OpenCTI, SOAR, data sovereignty, AI agent, cybersecurity

TABLA RESUMEN

	DATOS
Nombre y apellidos:	Manuel Araújo Baño
Título del proyecto:	Diseño e implementación de una plataforma corporativa de ciberinteligencia basada en OpenCTI para el soporte de un servicio de Cyber Threat Intelligence (CTI)
Directores del proyecto:	Ignacio Cea Fornies
El proyecto se ha realizado en colaboración de una empresa o a petición de una empresa:	NO
El proyecto ha implementado un producto: (esta entrada se puede marcar junto a la siguiente)	SI
El proyecto ha consistido en el desarrollo de una investigación o innovación: (esta entrada se puede marcar junto a la anterior)	SI
Objetivo general del proyecto:	Demostrar la viabilidad técnica y operativa de un servicio de Cyber Threat Intelligence mediante la centralización, correlación y enriquecimiento de información de amenazas en un entorno corporativo

Índice

RESUMEN.....	3
ABSTRACT.....	4
TABLA RESUMEN.....	5
Capítulo 1. ANTECEDENTES / ESTADO DEL ARTE.....	9
1.1 Introducción.....	9
1.2 Estado del arte.....	9
1.3 Contexto y justificación.....	10
1.4 Planteamiento del problema.....	11
Capítulo 2. OBJETIVOS.....	12
2.1 Objetivos generales.....	12
2.2 Objetivos específicos.....	12
2.3 Beneficios del proyecto.....	13
Capítulo 3. DESARROLLO DEL PROYECTO.....	14
3.1 Planificación del proyecto.....	14
3.1.1 Metodología de trabajo y herramientas de gestión.....	14
3.1.2 Fases del proyecto y esfuerzo estimado.....	14
3.1.3 Cronograma.....	15
3.2 Descripción de la solución, metodologías y herramientas empleadas.....	15
3.2.1 Infraestructura física y capa de virtualización.....	16
3.2.2 OpenCTI.....	18
3.2.3 Detección y Telemetría.....	21
3.2.4 Seguridad perimetral y Defensa Activa.....	22
3.2.5 Orquestación y Automatización.....	24
A. Alternativas evaluadas y justificación de la decisión.....	24
B. Descripción técnica de los flujos de trabajo.....	25
C. Desafíos técnicos y aprendizaje sobre la fragilidad de la automatización.....	28
3.3 Recursos requeridos.....	29
3.4 Presupuesto.....	30
3.4.1 ¿Por qué se elaboran dos presupuestos?.....	30
3.4.2 ¿Por qué el coste final depende de la infraestructura de implantación?.....	30
3.4.3 Mapeo del stack: laboratorio frente a edición enterprise.....	31
3.4.4 Presupuesto 1 — Laboratorio (TFG).....	32
3.4.5 Presupuesto 2 - Empresa simulada (mediana, ~500 empleados).....	34
3.5 Viabilidad.....	37
3.5.1 Viabilidad económica.....	37
3.5.2 Sostenibilidad a futuro.....	37
3.5.3 Contribución a la sostenibilidad.....	38

3.6 Resultados del proyecto.....	38
3.6.1 Resultados conforme a los objetivos específicos.....	39
3.6.2 Plan de pruebas y casos de uso validados.....	40
3.6.3 Desviaciones respecto a los objetivos iniciales.....	41
Capítulo 4. DISCUSIÓN.....	43
4.1 Limitaciones de la solución y de la tecnología empleada.....	43
4.2 Reflexión sobre la metodología empleada.....	43
4.3 Impacto y aportación del proyecto.....	43
Capítulo 5. CONCLUSIONES.....	44
5.1 Conclusiones del trabajo.....	44
5.2 Conclusiones personales.....	45
Capítulo 6. FUTURAS LÍNEAS DE TRABAJO.....	46
Capítulo 7. REFERENCIAS.....	47
Capítulo 8. ANEXOS.....	50

Índice de Figuras

Figura 1: Cronograma.....	15
Figura 2: Máquinas proxmox.....	16
Figura 3: Dashboard OpenCTI.....	18
Figura 4: Dashboard Wazuh.....	21
Figura 5: Dashboard OPNsense.....	23
Figura 6: Workflow SOAR.....	26
Figura 7: Workflow IA Agent.....	27
Figura 8: Red Lab.....	32
Figura 9: Red empresa 1.....	34
Figura 10: Red empresa 2.....	34

Índice de Tablas

Tabla 1: Mapeo Stack.....	32
Tabla 2: presupuesto lab.....	34
Tabla 3: presupuesto empresa software.....	36
Tabla 4: presupuesto empresa Capex & Opex & Software.....	37

Capítulo 1. ANTECEDENTES / ESTADO DEL ARTE

1.1 Introducción

La ciberseguridad ha evolucionado en los últimos años desde un enfoque perimetral y preventivo hacia un modelo basado en la detección y la respuesta ante incidentes. La sofisticación creciente de los adversarios y la complejidad de las infraestructuras corporativas han dejado obsoletas las defensas tradicionales por sí solas, dando paso a la Ciberinteligencia de Amenazas o Cyber Threat Intelligence (CTI), una disciplina cuyo objetivo es transformar datos dispersos sobre amenazas en conocimiento útil que permita anticiparse a los ataques.

Para comprender el alcance del presente trabajo conviene presentar tres conceptos sobre los que se asienta cualquier servicio CTI moderno. El primero es el de Indicador de Compromiso (IoC), un artefacto técnico observable —una dirección IP, un hash, un dominio— que sugiere la presencia de actividad maliciosa. El segundo es STIX (Structured Threat Information Expression), un estándar abierto que representa la información de amenazas no como listas planas, sino como un grafo de objetos relacionados entre sí (un IoC vinculado a un malware, este a una campaña, y la campaña a un grupo adversario), complementado por el protocolo TAXII (Trusted Automated Exchange of Intelligence Information) para su intercambio entre organizaciones. El tercero es la distinción entre acumulación de datos y generación de contexto: el valor real de la ciberinteligencia no reside en el volumen de IoCs recibidos, sino en la capacidad de relacionarlos, priorizarlos y vincularlos con la realidad operativa de la organización.

Con este marco conceptual establecido, el siguiente apartado analiza el estado actual de las soluciones tecnológicas que las organizaciones emplean para implementar servicios de ciberinteligencia.

1.2 Estado del arte

El panorama actual de la ciberseguridad se caracteriza por un desequilibrio entre la capacidad de ataque de los adversarios y la capacidad de respuesta de las organizaciones. Según el informe *Cost of a Data Breach Report 2024* de IBM Security, el coste medio de una brecha de datos ha alcanzado un récord histórico de 4,88 millones de dólares a nivel global. Más preocupante aún son los tiempos de gestión: las organizaciones tardan una media de 194 días en identificar un incidente y 64 días adicionales en contenerlo (IBM Security, 2024).

En cuanto a las tecnologías de defensa, el estado del arte se divide principalmente en tres categorías:

1. **Sistemas de gestión de eventos (SIEM):** Herramientas como Splunk o Wazuh (Wazuh, Inc., s.f.) que centralizan los logs. Aunque son efectivas para la correlación de eventos internos, a menudo carecen de contexto externo sobre quién ataca y por qué.
2. **Plataformas de Protección de Endpoint (EDR (Endpoint Detection and Response) / XDR):** Soluciones como CrowdStrike Falcon o Microsoft Defender for Endpoint, centradas en la detección conductual en los dispositivos finales. Aunque ofrecen una visibilidad detallada del

endpoint, su alcance se limita a los equipos donde está instalado el agente, sin aportar visión sobre el contexto de la amenaza más allá del propio dispositivo.

3. **Plataformas de Intercambio de Inteligencia (TIP):** Aquí destacan soluciones de código abierto como MISP (*Malware Information Sharing Platform*). Si bien MISP es el estándar de facto para compartir listas de indicadores (hashes, direcciones IP), su modelo de datos es a menudo plano, dificultando la visualización de relaciones complejas entre campañas, actores y vulnerabilidades.

El estándar técnico predominante para la representación de esta información es STIX 2.1 (OASIS Cyber Threat Intelligence Technical Committee, 2021), que permite modelar las amenazas como objetos relacionados. Sin embargo, muchas implementaciones actuales no aprovechan todo el potencial de este modelo gráfico, limitándose a la ingesta masiva de listas negras que saturan los sistemas de defensa.

1.3 Contexto y justificación

La motivación principal para el desarrollo de este proyecto reside en el fenómeno conocido como "Fatiga de Alertas". Los analistas de seguridad reciben diariamente miles de notificaciones de sus sistemas SIEM e IDS, muchas de las cuales carecen de contexto. Esto provoca que amenazas reales pasen desapercibidas entre una multitud de falsos positivos (IBM, s.f.).

Este proyecto justifica su relevancia en la necesidad de pasar de una postura reactiva a una proactiva mediante el uso de **OpenCTI** (OpenCTI Platform, s.f.). A diferencia de otras soluciones, **OpenCTI** utiliza una base de datos orientada a grafos que permite correlacionar automáticamente información técnica (por ejemplo, un hash de malware) con contexto estratégico (el grupo criminal que lo utiliza y sus tácticas).

Adicionalmente, existe una justificación normativa y de soberanía del dato. Gran parte de las soluciones de CTI en la nube son provistas por empresas estadounidenses, lo que puede implicar conflictos con normativas europeas como el RGPD (Parlamento Europeo y Consejo de la Unión Europea, 2016) o la exposición a leyes extraterritoriales como la CLOUD Act estadounidense, que faculta a las autoridades de Estados Unidos a requerir datos almacenados por proveedores estadounidenses incluso cuando dichos datos residen físicamente fuera de su territorio (U.S. Congress, 2018). Por ello, este trabajo propone y justifica un despliegue *on-premise* (en un proveedor europeo, OVHcloud) sobre infraestructura virtualizada con Proxmox, garantizando que el ciclo de inteligencia permanezca bajo control directo de la organización y dentro del marco legal europeo.

1.4 Planteamiento del problema

El problema central que aborda este Trabajo de Fin de Grado es la **fragmentación y la falta de contexto en los sistemas de seguridad corporativos**.

Actualmente, las herramientas de seguridad funcionan a menudo como silos de información: el firewall bloquea IPs, el EDR detecta procesos y el SIEM almacena logs, pero rara vez estos sistemas comparten un "cerebro" común que entienda la naturaleza de la amenaza. Las soluciones comerciales que prometen esta unificación suelen tener costes prohibitivos para muchas empresas, y las soluciones de código abierto tradicionales requieren una integración compleja que no siempre está documentada para entornos productivos.

Por tanto, el problema técnico a resolver es: ¿Cómo diseñar e implementar una plataforma de ciberinteligencia corporativa, basada en software libre y soberanía de datos, que sea capaz de ingerir, normalizar y distribuir inteligencia procesable a los sistemas defensivos (SIEM y firewall) para reducir el tiempo de respuesta ante incidentes?

Capítulo 2. OBJETIVOS

2.1 Objetivos generales

El objetivo general del presente trabajo consiste en diseñar, implementar y validar un prototipo funcional de una plataforma corporativa de ciberinteligencia *on-premise* basada en OpenCTI, capaz de centralizar la gestión de amenazas y orquestar automáticamente la distribución de inteligencia hacia sistemas de defensa y monitorización perimetral e interna.

Con ello, se pretende demostrar cómo una arquitectura basada en software libre puede transformar un conjunto de datos aislados en conocimiento accionable, reduciendo los tiempos de respuesta ante incidentes mediante la automatización de procesos.

2.2 Objetivos específicos

Para alcanzar el objetivo general, el proyecto se divide en los siguientes objetivos específicos, los cuales abarcan desde el despliegue de la infraestructura hasta su validación funcional:

- Desplegar una infraestructura virtualizada soberana sobre Proxmox VE que aloje de forma aislada y bajo control directo los componentes del proyecto.
- Implementar la plataforma OpenCTI con sus dependencias y desarrollar conectores propios de ingesta y enriquecimiento adaptados al entorno.
- Integrar Wazuh como SIEM/XDR para la monitorización interna y OPNsense como firewall para la protección perimetral.
- Desarrollar dos flujos de orquestación SOAR sobre n8n: uno automático para procesar alertas de Wazuh, y otro conversacional para que el analista ejecute acciones mediante lenguaje natural.
- Validar la arquitectura mediante casos de uso simulados que comprueben el funcionamiento del ciclo de inteligencia completo.

2.3 Beneficios del proyecto

La implementación de esta plataforma aporta una serie de ventajas operativas y estratégicas al entorno corporativo simulado, entre las que destacan:

- **Reducción de la "Fatiga de Alertas":** Al utilizar OpenCTI como núcleo de conocimiento, las alertas generadas por sistemas como Wazuh se enriquecen con contexto táctico y estratégico. Esto permite a los analistas discriminar rápidamente entre falsos positivos y amenazas reales, optimizando el proceso de triaje.
- **Respuesta automatizada y ágil:** La integración de n8n permite pasar de un modelo de seguridad reactivo, dependiente de la intervención manual continua, a un modelo proactivo. Las alertas se notifican en tiempo real vía Discord con propuestas de mitigación concretas, disminuyendo drásticamente la ventana de exposición de la organización.
- **Soberanía de los datos y cumplimiento normativo:** Al apostar por una arquitectura *on-premise* alojada en infraestructura bajo control directo (Proxmox), se evita la dependencia de nubes públicas sujetas a normativas extraterritoriales. Esto asegura que la gestión de datos sensibles de seguridad cumpla de manera estricta con las normativas de privacidad europeas.
- **Eficiencia en el uso de recursos:** El proyecto demuestra que es posible desplegar una plataforma avanzada de CTI y orquestación utilizando soluciones de código abierto. Las decisiones arquitectónicas, como la selección estratégica de *feeds* de inteligencia (priorizando AlienVault para reducir la carga de Elasticsearch) y el manejo de cuotas en APIs de enriquecimiento, prueban la viabilidad de implementar estas soluciones en organizaciones con recursos limitados.

Capítulo 3. DESARROLLO DEL PROYECTO

3.1 Planificación del proyecto

3.1.1 Metodología de trabajo y herramientas de gestión

Dada la naturaleza técnica y de integración de sistemas de este proyecto, se ha descartado un modelo de desarrollo puramente secuencial (en cascada). En su lugar, se ha optado por una **metodología iterativa e incremental basada en componentes**.

Este enfoque ha permitido abordar el ecosistema de forma modular: cada componente de la arquitectura (Proxmox, OpenCTI, OPNsense, Wazuh y n8n) se ha tratado como un módulo independiente. El flujo de trabajo consistía en desplegar un sistema, realizar su configuración base, estabilizar y proceder al siguiente. Durante la fase de integración, este enfoque iterativo resultó fundamental, ya que permitía retroceder a configuraciones anteriores para realizar los ajustes necesarios (por ejemplo, reasignación de recursos en la virtualización tras evaluar el consumo real de los conectores de OpenCTI).

3.1.2 Fases del proyecto y esfuerzo estimado

En el anteproyecto original se estimó una dedicación inicial de 300 horas. A lo largo del desarrollo, la distribución de este tiempo ha sufrido ciertas variaciones, propias de la naturaleza práctica del trabajo, requiriendo un ajuste en el esfuerzo dedicado a cada etapa. El proyecto se ha estructurado finalmente en las siguientes cinco fases, sumando un total aproximado de 325 horas:

- **Fase 1. Investigación y Estado del Arte (40 horas):** Dedicada a la revisión bibliográfica, la familiarización del estándar STIX 2.1, la comparativa de plataformas y el análisis de la normativa sobre ciberinteligencia.
- **Fase 2. Diseño de la Arquitectura (15 horas):** Definición topológica y selección del stack tecnológico. El esfuerzo real en esta fase fue inferior al estimado inicialmente. Al tratarse de un despliegue desde cero sobre Proxmox VE, la comunicación entre máquinas virtuales resultó directa y se optó por mantener los puertos predeterminados de cada servicio. Se asume que este esfuerzo habría sido considerablemente mayor en un escenario de integración sobre una infraestructura empresarial preexistente o legacy.
- **Fase 3. Despliegue e Implementación (170 horas):** Ha sido la fase más extensa y la que mayor capacidad de resolución de problemas ha demandado. El incremento de horas se justifica por varios obstáculos técnicos encontrados durante el despliegue:
 - **Desarrollo de conectores:** La creación del conector *Multi-Feed (drb-ra, s.f.)* requirió un esfuerzo significativo para mapear y normalizar correctamente la información externa hacia objetos del estándar STIX.
 - **Despliegue de Wazuh:** Se experimentaron problemas de permisos e incompatibilidades al intentar desplegar el stack de Wazuh mediante contenedores Docker (Docker, Inc., s.f.), lo que obligó a replantear la estrategia y realizar una instalación local sobre el sistema operativo.

- **Orquestación en n8n:** La programación lógica de los *workflows* para la automatización de respuestas requirió múltiples iteraciones para garantizar la correcta interconexión entre las APIs.
- **Depuración de OpenCTI:** Se identificó un error de renderizado en la interfaz gráfica que provocaba fallos de visualización. Tras la fase de *troubleshooting*, se diagnosticó como un *bug* específico de la versión desplegada, documentado únicamente en incidencias de repositorios en GitHub, lo que exigió actualizar la versión.
- **Fase 4. Integración, Pruebas y Validación (40 horas):** Comprobación del ciclo de inteligencia completo. Se validaron los flujos desde la ingesta de Indicadores de Compromiso (IoC) hasta la generación de alertas mitigadoras vía Discord a través de n8n.
- **Fase 5. Documentación y Memoria (60 horas):** Fase final enfocada en la redacción de la presente memoria académica, la organización de los anexos técnicos (configuraciones detalladas recogidas en Obsidian) y la preparación del material para la defensa pública.

3.1.3 Cronograma

A continuación, se presenta el diagrama de Gantt que ilustra la distribución temporal de las fases descritas anteriormente a lo largo de los meses de desarrollo del Trabajo de Fin de Grado, reflejando el solapamiento necesario entre la implementación, las pruebas y la redacción de la documentación.

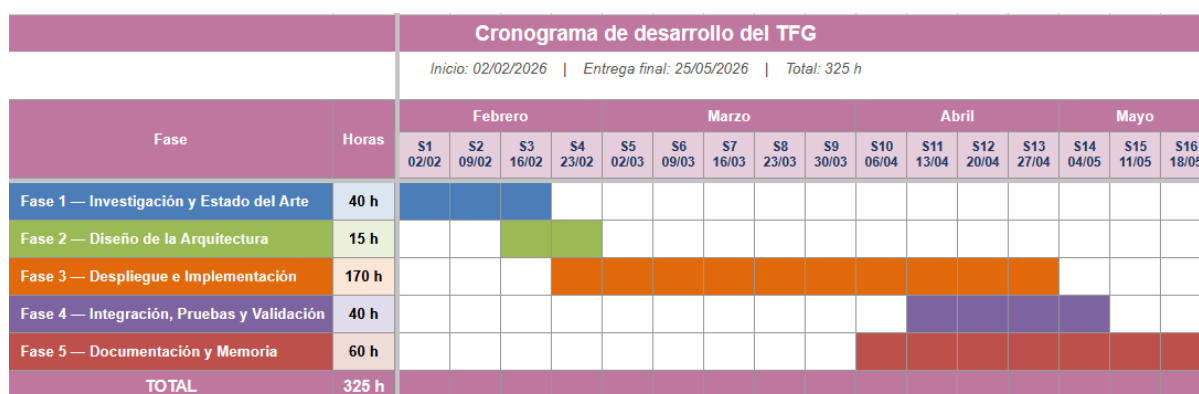


Figura 1: Cronograma

3.2 Descripción de la solución, metodologías y herramientas empleadas

El diseño y desarrollo de la plataforma se ha regido por una arquitectura modular, estructurada en cinco pilares fundamentales: infraestructura base y capa de virtualización, núcleo de ciberinteligencia, sistema de detección y telemetría, seguridad perimetral y defensa activa, y orquestación automatizada. A continuación, se detallan las decisiones técnicas e implementaciones llevadas a cabo en cada uno de estos bloques.

3.2.1 Infraestructura física y capa de virtualización

El primer nivel de la arquitectura lo constituye la capa física y el hipervisor. En proyectos de ciberinteligencia, la elección del entorno donde residen los datos es crítica, ya que determina no solo el rendimiento, sino también la jurisdicción legal y el control técnico sobre la infraestructura.

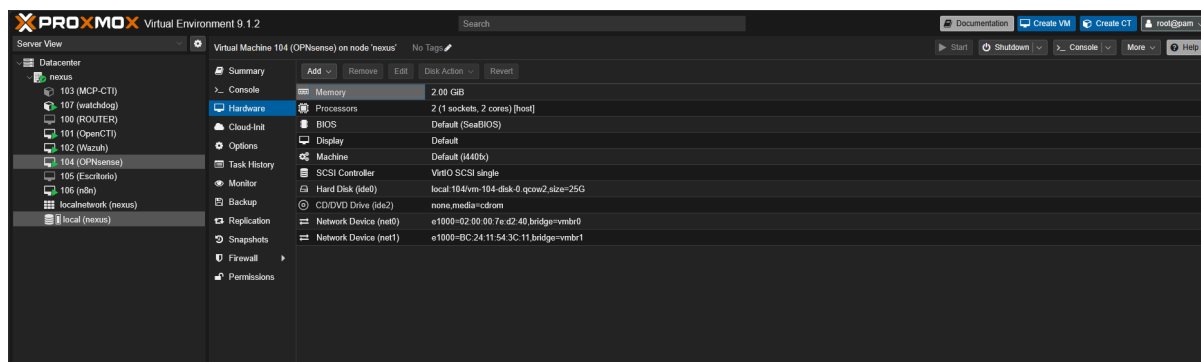


Figura 2: Máquinas proxmox

A. Alternativas evaluadas y selección del entorno

Durante la fase de diseño se evaluaron dos alternativas principales para el alojamiento de la plataforma:

1. **Cloud Público (AWS, Azure, GCP):** Se consideró el despliegue mediante instancias virtualizadas en proveedores de nube pública. Si bien ofrecen una alta escalabilidad, se descartaron por dos motivos fundamentales: el coste elevado de mantener instancias con los requisitos de RAM de OpenCTI y, principalmente, por la **limitación de soberanía**. Bajo la legislación estadounidense (*U.S. CLOUD Act*), los datos almacenados en estos proveedores pueden estar sujetos a requerimientos gubernamentales extranjeros, lo que entra en conflicto con el objetivo de este TFG de diseñar una solución soberana.
2. **Infraestructura Local (on-premise / bare metal):** Esta opción permite un control total sobre el hardware. Dentro de esta categoría, se seleccionó un servidor **bare metal de OVHcloud**.

La decisión por **OVHcloud** se justifica por su cumplimiento estricto con el Reglamento General de Protección de Datos (**RGPD**) y por ser un proveedor europeo, garantizando que la inteligencia de amenazas generada no salga de la jurisdicción de la Unión Europea. Además, el acceso **bare metal** permite interactuar directamente con la CPU y la memoria, optimizando el rendimiento de las bases de datos NoSQL y de búsqueda (Elasticsearch) que componen el núcleo del sistema.

B. Hipervisor: Selección de Proxmox VE

Para la gestión de los recursos físicos, se ha optado por **Proxmox Virtual Environment (VE)** (Proxmox Server Solutions, s.f.) frente a soluciones propietarias como VMware vSphere. Esta elección responde a criterios de eficiencia y coste:

- **Naturaleza abierta:** Al ser una solución basada en Debian, permite una personalización profunda del sistema operativo host, algo fundamental para aplicar parches de kernel necesarios para la estabilidad de los contenedores de seguridad.
- **Licenciamiento:** Permite disponer de una plataforma de virtualización de nivel empresarial (KVM/LXC) sin incurrir en costes de licencia, lo cual es coherente con el enfoque del proyecto de utilizar herramientas de código abierto para reducir el presupuesto corporativo.
- **Capacidades de respaldo y recuperación:** Proxmox proporciona de forma nativa la creación de snapshots de máquinas virtuales, que en el laboratorio se han utilizado como mecanismo básico de recuperación ante cambios críticos de configuración. En un despliegue corporativo, esta capacidad se complementará con backup off-site mediante Proxmox Backup Server (PBS), garantizando la restauración de las VMs ante fallos catastróficos del hardware o del sitio principal.

C. Implementación técnica y acceso a la red

Una vez instalado el hipervisor Proxmox sobre el servidor físico (asignando los 16 vCores y 64 GB de RAM disponibles), surgió el primer reto de diseño: la gestión del acceso remoto. Dado que el servidor solo disponía de una interfaz de red pública y el objetivo era crear un laboratorio aislado en la red **10.0.0.0/24**, fue necesario implementar una solución de enrutamiento temporal.

Para ello, se desplegó una **Máquina Virtual de Enrutamiento inicial** (basada en Debian 12). La configuración técnica de este nodo fue crítica para la viabilidad del proyecto en sus etapas tempranas:

- **Configuración de Red:** Se configuraron dos interfaces (*Network Bridges*), una externa conectada a la WAN y otra interna (**vmbbr1**) para el segmento del laboratorio.
- **Lógica de Forwarding:** Se habilitó el reenvío de paquetes a nivel de kernel (**net.ipv4.ip_forward=1**) y se implementó un conjunto de reglas mediante **nftables**.
- **Acceso Seguro:** Este nodo actuó como bastión (*Jump Server*), permitiendo el acceso mediante SSH y NAT hacia el resto de las máquinas del segmento local que aún no tenían salida directa a internet.

Los detalles específicos de los comandos de configuración de **netplan**, las reglas de enmascaramiento de red (**masquerade**) y los scripts de persistencia de **nftables** se encuentran documentados en el anexo (**ver Anexo - Configuración VM Router**).

Este enrutador inicial cumplió su función de estabilización del entorno, permitiendo el despliegue de los servicios pesados. Sin embargo, su naturaleza era puramente funcional. Para alcanzar los requisitos de seguridad perimetral y segmentación profesional exigidos por un servicio de CTI, este componente servirá de base para la transición hacia el firewall corporativo **OPNsense**, el cual asumirá las funciones de VPN y filtrado avanzado que se describirán más adelante.

3.2.2 OpenCTI

El componente central y "cerebro" de la infraestructura propuesta es la plataforma de ciberinteligencia. Su función principal es centralizar, estructurar y almacenar toda la información relativa a ciberamenazas, sirviendo como única fuente de verdad (*Single Source of Truth*) para el resto de los sistemas defensivos del entorno corporativo.

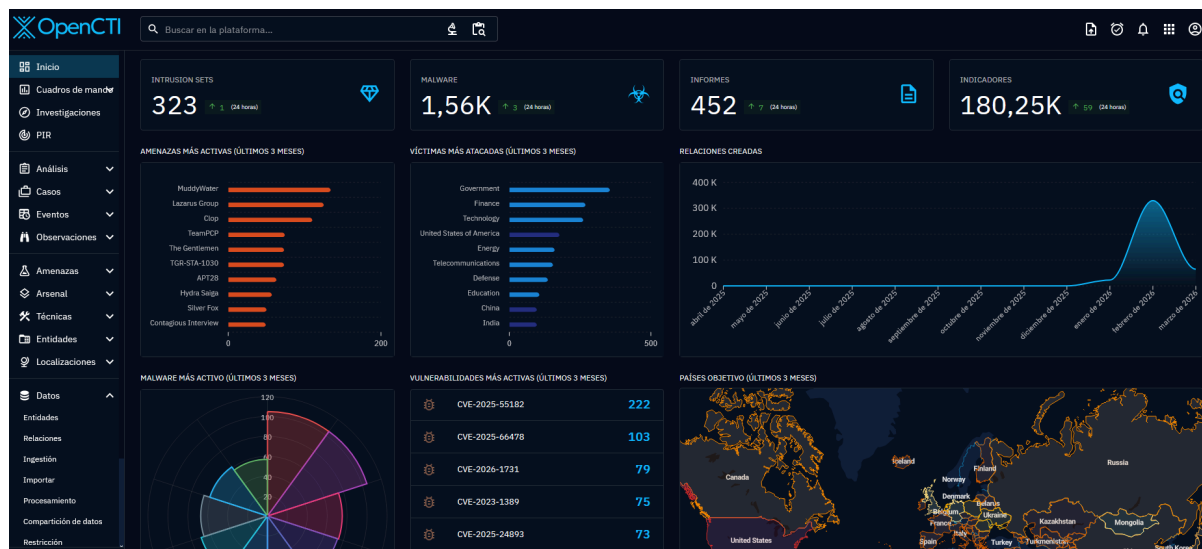


Figura 3: Dashboard OpenCTI

A. Alternativas evaluadas y justificación de la decisión

Durante la fase de diseño, se evaluaron diferentes plataformas TIP (*Threat Intelligence Platform*). La principal alternativa considerada fue **MISP** (*Malware Information Sharing Platform*), al ser el estándar de facto en la comunidad de código abierto. Sin embargo, **MISP** presenta un enfoque centrado fundamentalmente en compartir listas planas de indicadores (hashes, direcciones IP o dominios), careciendo de capacidades nativas para modelar relaciones de forma visual e intuitiva.

Se optó por **OpenCTI** (Filigran, s.f.) debido a su arquitectura basada en grafos y su adopción nativa del estándar **STIX 2.1**. A diferencia de modelos más simples, OpenCTI permite correlacionar de manera automática un Indicador de Compromiso (IoC) técnico con la vulnerabilidad que explota, la campaña a la que pertenece y el grupo de amenaza avanzada persistente (APT) que lo utiliza. Esta capacidad de dotar de contexto estratégico a los datos técnicos es el requisito fundamental que justifica su elección para este proyecto.

B. Descripción técnica del despliegue

OpenCTI no es una aplicación monolítica, sino un ecosistema complejo que requiere múltiples servicios subyacentes operando en conjunto: una base de datos de búsqueda y analítica (**Elasticsearch**), un motor de caché y estados (**Redis**), un sistema de almacenamiento de objetos (**MinIO**) y un *broker* de mensajería para las colas de procesamiento (**RabbitMQ**).

Dada esta complejidad, se descartó la instalación directa sobre el sistema operativo y se optó por un despliegue contenerizado mediante **Docker y Docker Compose**. Esta decisión permitió levantar y gestionar todo el *stack* tecnológico de forma declarativa, aislada y reproducible, simplificando tanto el despliegue inicial como el mantenimiento.

Durante la implementación de la infraestructura base, fue necesario aplicar optimizaciones a nivel del *kernel* del servidor anfitrión (Debian). Específicamente, se modificó el parámetro **vm.max_map_count** a un valor de **262144**. Este ajuste fue crítico, ya que de omitirse, los requerimientos de mapeo de memoria de Elasticsearch provocan la caída inmediata del contenedor en el arranque (**detallado en el Anexo - OpenCTI**).

Adicionalmente, para optimizar las operaciones de mantenimiento, se desarrolló un **script** de administración personalizado en Bash. Esta herramienta permite gestionar el ciclo de vida de los contenedores separando lógicamente la capa "Core" (bases de datos y plataforma) de la capa de "Conectores" (workers), evitando reinicios innecesarios de las bases de datos cuando únicamente se requiere actualizar la lógica de ingesta de un feed (**ver Anexo - opencti-control**).

En cuanto a la gestión de credenciales (claves de API de los conectores externos, tokens de administración de OpenCTI, contraseñas, etc...), todas se externalizan en un fichero **“.env”** referenciado por las variables de entorno del Docker Compose. Este enfoque mantiene los secretos fuera del código fuente y permite rotarlos sin modificar la definición de los servicios.

C. Estrategia y desarrollo de conectores

Para demostrar la capacidad de integración de la plataforma, el proyecto contempló el diseño y desarrollo de tres conectores propios que abarcan los tres flujos de datos que soporta OpenCTI, de los cuales dos llegaron a desplegarse en producción y un tercero se diseñó pero no se puso en operación. Adicionalmente, durante la fase inicial se usaron más conectores oficiales para nutrir el sistema, pero algunos se retiraron posteriormente por la saturación del equipo:

1. **Conector de Ingesta Externa (Multi-Feed):** Desarrollado en Python, este conector consulta periódicamente cuatro fuentes OSINT de inteligencia abierta: dos **feeds** JSON de infraestructura C2 procedentes del repositorio C2IntelFeeds (drb-ra, s.f.) —específicamente los listados de servidores Cobalt Strike y PoshC2—, el **feed** de URLs de **phishing** de OpenPhish (OpenPhish, s.f.) y la lista oficial de nodos de salida de la red Tor (The Tor Project, s.f.). El conector parsea los formatos heterogéneos de cada fuente y los transforma en objetos STIX 2.1. Su implementación fue un éxito, operando de forma estable al consumir únicamente fuentes públicas sin necesidad de claves API ni autenticación (**ver Anexo - *multifeed connector documentación***). A este desarrollo propio se sumaron conectores oficiales base, como el de **AlienVault OTX** (AT & T Cybersecurity, s.f.), para dotar a la plataforma de un volumen realista de inteligencia.
2. **Conector de Enriquecimiento Interno (Gemini):** Se diseñó un conector que, bajo demanda, envía la información de un objeto STIX a un modelo de lenguaje (LLM) de Google Gemini para generar un análisis técnico en formato *Markdown*. Aunque el desarrollo técnico es

plenamente funcional e interactúa correctamente con la API de OpenCTI para crear notas enriquecidas, su operatividad en el laboratorio quedó degradada ("quota exceeded"). Esto se debe a las estrictas limitaciones de cuota de la capa gratuita de *Google AI for Developers API*. En un entorno corporativo de producción, esta limitación se subsana mediante la contratación de cuotas empresariales (**ver Anexo - *gemini connector documentación***).

3. **Conector de Stream:** Se desarrolló la estructura base de un conector de flujo en tiempo real (**stream**). Sin embargo, se decidió no desplegarlo en la arquitectura final del laboratorio, al no contar con un sistema consumidor intermedio que requiriese un consumo persistente de datos mediante este protocolo y no poder probar su funcionamiento. A los conectores propios anteriores se sumaron varios conectores oficiales mantenidos por la comunidad de OpenCTI. Junto al ya mencionado de AlienVault OTX, se utilizó el conector de MITRE ATT&CK (MITRE Corporation, s.f.) para ingerir el conjunto completo de tácticas, técnicas y procedimientos (TTPs) catalogados por dicho **framework**, que sirven de base estructural para la correlación de los IoCs con comportamientos de actores conocidos.

D. Resolución de problemas

Durante la fase de integración, se detectó un problema crítico a nivel de interfaz de usuario (*frontend*): la información de los vértices y relaciones de los objetos no se renderizaba correctamente en los grafos interactivos. Tras una investigación técnica y la consulta en repositorios de código, se identificó que se trataba de un *bug* reconocido de la versión inicialmente desplegada ([Issue loading a report in Firefox browser in both Linux & Android #14025](#)). El incidente se solventó sin impacto en los datos mediante la actualización de la imagen de Docker a la versión de parche correspondiente, demostrando la agilidad del modelo basado en contenedores.

Con OpenCTI plenamente operativo e ingiriendo inteligencia de fuentes abiertas, el sistema dispone de un repositorio centralizado de amenazas. Sin embargo, para que esta inteligencia aporte un valor defensivo real, debe ser consumida y accionada por los sistemas encargados de la vigilancia de la red. Esta función la asumen **Wazuh** y **OPNsense**, las herramientas responsables de aplicar esta inteligencia en la monitorización de *endpoints* y la protección del perímetro, respectivamente.

3.2.3 Detección y Telemetría

Para dotar a la infraestructura de capacidades de monitorización interna y detección de amenazas, se ha integrado Wazuh como solución SIEM (*Security Information and Event Management*) y XDR (*Extended Detection and Response*). Este componente actúa como el sensor principal de la red, recogiendo telemetría de las máquinas críticas mediante agentes y procesándola para identificar actividades anómalas.

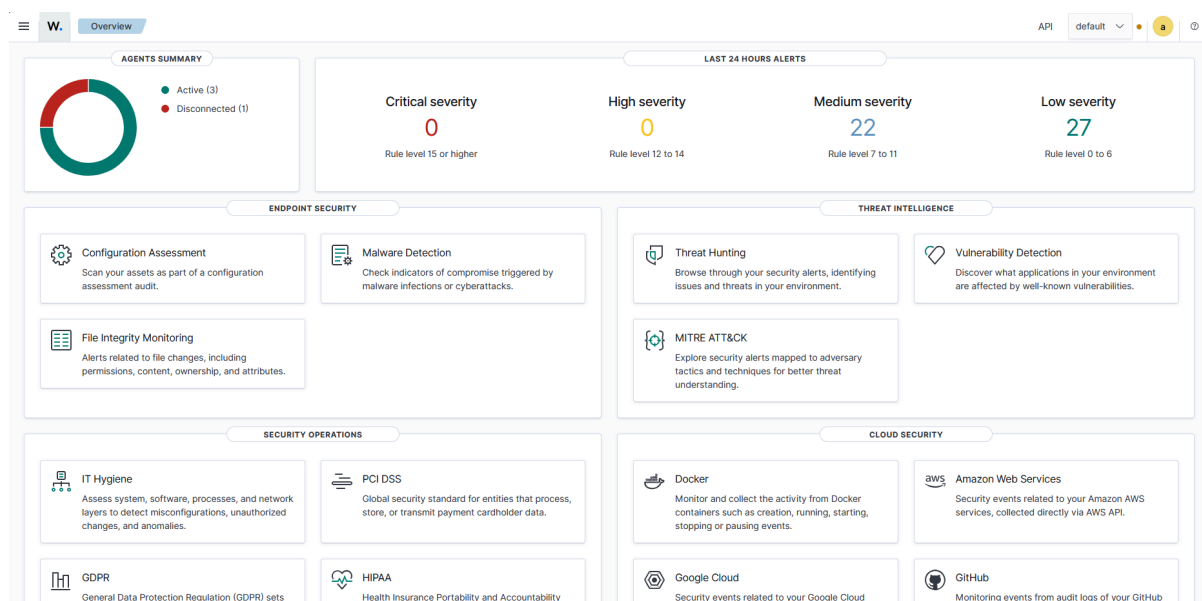


Figura 4: Dashboard Wazuh

A. Alternativas evaluadas y justificación de la decisión

Inicialmente, en coherencia con la arquitectura del núcleo de ciberinteligencia, se evaluó y proyectó la contenerización completa de la plataforma Wazuh utilizando Docker. Sin embargo, durante la fase práctica de implementación, se evidenció la inviabilidad operativa de este modelo para este componente específico.

La principal fricción técnica surgió de la gestión de permisos en los volúmenes persistentes y de la estricta infraestructura de clave pública (**PKI**) interna de la herramienta. Wazuh requiere una Autoridad de Certificación propia para emitir y validar los certificados TLS que cifran la comunicación bidireccional entre los agentes y el nodo central (*manager*). Las capas de abstracción de red y almacenamiento propias de Docker generaban conflictos constantes en el acceso a estos certificados y en la validación del *handshake*.

Ante esta inestabilidad, se tomó la decisión estratégica de abandonar el despliegue contenerizado en favor de una instalación nativa sobre el sistema operativo Debian. Este cambio de paradigma resultó

fundamental, ya que proporcionó el control absoluto sobre el sistema de archivos local y la gestión de procesos nativos, asegurando la robustez de las comunicaciones cifradas de la plataforma.

B. Descripción técnica del despliegue

La implementación definitiva se llevó a cabo utilizando la arquitectura *All-in-One* proporcionada por el asistente de instalación oficial. Este modelo despliega e interconecta localmente los componentes de la herramienta: la base de datos (**wazuh-indexer**), el motor de correlación y recolección (**wazuh-manager**) y la interfaz gráfica (**wazuh-dashboard**).

Para soportar la carga de procesamiento, se asignaron 4 vCores, 8 GB de memoria RAM y 80GB de almacenamiento a la máquina virtual dedicada.

Una vez estabilizado el nodo central, se procedió a la instalación de los agentes de monitorización en los servidores críticos del segmento de red del laboratorio. Toda la configuración técnica se encuentra detallada en el **Anexo - Wazuh**.

Con los agentes desplegados y el manager operativo, la arquitectura cuenta con una fuente fiable de eventos de seguridad. No obstante, la generación de alertas por sí sola es insuficiente para un modelo proactivo y contribuye a la "fatiga de alertas" mencionada en la justificación del proyecto. Para que esta telemetría alcance su verdadero valor, debe ser interceptada, procesada y enriquecida automáticamente con la inteligencia proveniente de OpenCTI. Este rol de "pegamento" lógico y automatización defensiva es asumido por la plataforma **n8n**, actuando como orquestador SOAR, como veremos más adelante

3.2.4 Seguridad perimetral y Defensa Activa

El perímetro de la infraestructura está protegido y gestionado por OPNsense, que actúa como *gateway* principal de la red del laboratorio. Este componente no solo ejerce las funciones tradicionales de enrutamiento y filtrado de paquetes entre segmentos de red, sino que constituye el punto de ejecución de la defensa activa, consumiendo listas de bloqueo dinámicas para aislar la red de amenazas conocidas. Además, proporciona un canal de administración remota seguro mediante VPN.

A. Alternativas evaluadas y justificación de la decisión

Para cubrir el rol de firewall perimetral, se evaluaron los dos candidatos de código abierto más relevantes del sector: **pfSense** y **OPNsense** (OPNsense Project, s.f.)(ambos basados en FreeBSD). La decisión de adoptar OPNsense se fundamentó en dos criterios técnicos:

1. **Sostenibilidad del ecosistema con código abierto:** A nivel estratégico, el desarrollo de pfSense (por parte de la empresa Netgate) ha mostrado desde 2021 una clara tendencia hacia la comercialización (*pfSense Plus*), reduciendo progresivamente las funcionalidades de su versión comunitaria. OPNsense, por el contrario, nació como un *fork* enfocado en garantizar un desarrollo abierto, transparente y con actualizaciones de seguridad semanales

documentadas públicamente. Para un proyecto que diseña una infraestructura corporativa basada en software libre, la viabilidad a largo plazo es un factor de diseño decisivo.

2. **Soporte nativo para Aliases dinámicos:** A nivel funcional, OPNsense destaca por gestionar “alias” de hosts que pueden modificarse en tiempo real mediante su API REST, sin necesidad de recargar el conjunto de reglas. Esta característica habilita un patrón de defensa activa más selectivo que el simple consumo pasivo de un **feed** externo: la capa de orquestación SOAR descrita en el apartado 3.2.5 aplica su propia lógica de análisis sobre cada alerta recibida y, sólo cuando una IP queda confirmada como maliciosa en el contexto concreto del incidente, el flujo invoca la API de OPNsense para añadirla al alias “**Bloqueo_SOAR**”. Una regla de firewall asociada bloquea de forma inmediata todo el tráfico cuyo origen coincida con cualquier IP listada en el alias, cerrando así el ciclo de defensa activa planteado en este proyecto. La configuración técnica del alias y de la regla se documenta en el anexo correspondiente a OPNsense.

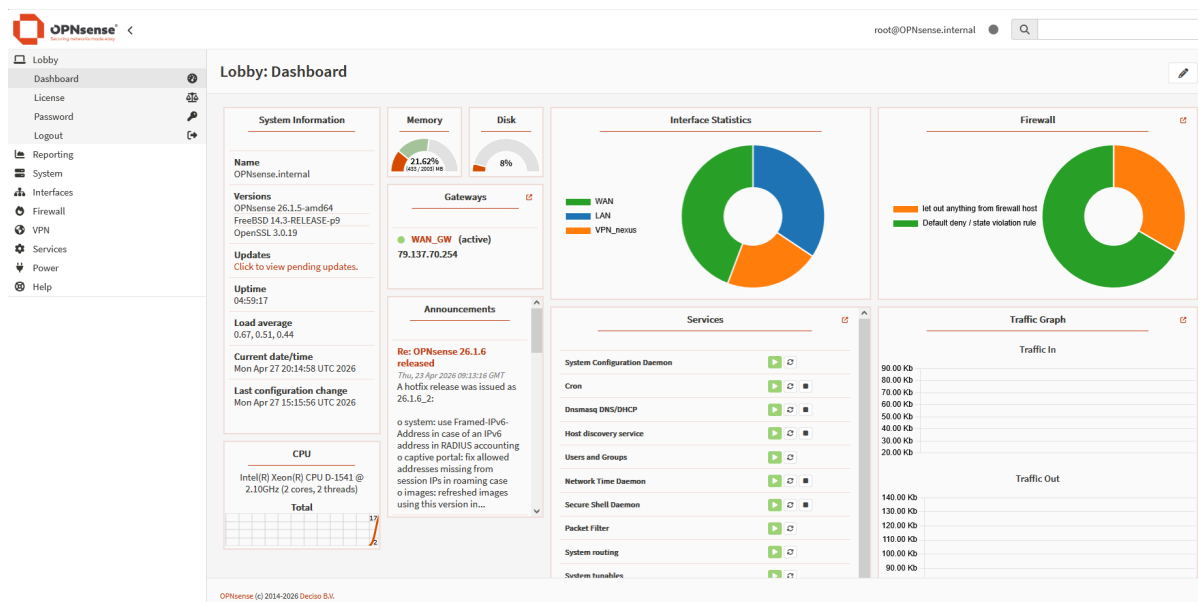


Figura 5: Dashboard OPNsense

En cuanto a la solución de acceso remoto al firewall, se evaluó **OpenVPN** frente a **WireGuard** (Donenfeld, 2017). Se optó por WireGuard debido a su arquitectura minimalista. La drástica reducción en la complejidad del código se traduce en una menor superficie de ataque y un rendimiento de cifrado superior, lo cual es coherente con los principios de diseño de una infraestructura orientada a la ciberseguridad.

B. Descripción técnica y resolución de problemas

El despliegue de OPNsense se realizó sobre una máquina virtual en Proxmox con 2 vCores, 2 GB de RAM y dos interfaces de red virtualizadas (**net0** para WAN y **net1** para LAN).

En primer lugar, se implementó el túnel **WireGuard** para garantizar un acceso administrativo seguro. La autenticación no se basa en certificados X.509 tradicionales, sino en un intercambio estricto de claves públicas asimétricas utilizando criptografía de Curva Elíptica (Curve 25519). Solo los pares cuyas claves públicas hayan sido previamente registradas en el firewall pueden establecer comunicación, descartando silenciosamente cualquier otra petición.

Durante la fase de configuración de red y resolución de nombres internos (DNS), surgió un desafío técnico crítico que obligó a rediseñar la topología lógica. Inicialmente, se pretendía utilizar el dominio interno **.nexus** para acceder a los servicios del laboratorio (por ejemplo, **opencti.nexus**). Sin embargo, se vio que **.nexus** es un dominio oficial (gTLD) incluido en las listas de precarga **HSTS (HTTP Strict Transport Security)** integradas en los navegadores modernos. Esto provocaba un bloqueo sin solución al intentar acceder mediante HTTP o con certificados TLS autofirmados.

Para solucionar este incidente de diseño, se habilitó el servicio **Unbound DNS** en OPNsense y se configuraron *Host Overrides*. Se adoptó el dominio inventado **.ryoiki**, el cual no colisiona con ningún TLD real ni está sujeto a políticas HSTS, permitiendo una resolución de nombres segura dentro del segmento LAN (**detallado en el Anexo - OPNsense**).

3.2.5 Orquestación y Automatización

El último pilar de la solución, y el que dota de inteligencia operativa a todo el conjunto, es la capa de orquestación. Actuando como punto de unión lógico del laboratorio, n8n asume el rol de plataforma SOAR (Security Orchestration, Automation and Response), permitiendo que la telemetría captada y la inteligencia almacenada se transformen en acciones defensivas concretas. Sobre esta capa se han desarrollado dos flujos complementarios: uno que reacciona ante alertas Wazuh con mínima intervención humana dependiendo de la alerta, y otro conversacional, que habilita al operador para iniciar acciones sobre la infraestructura mediante lenguaje natural a través de un agente IA con capacidad de invocación de herramientas.

A. Alternativas evaluadas y justificación de la decisión

Para cubrir las necesidades de automatización, se analizaron tres aproximaciones distintas: el uso de scripts personalizados en Python, la plataforma específica de ciberseguridad Shuffle y la herramienta de propósito general n8n.

Frente a scripts en Python: Aunque el desarrollo a medida ofrece flexibilidad total, su mantenimiento en un entorno corporativo es costoso. Cada integración requeriría construir desde cero la lógica de autenticación, gestión de errores e integración de las distintas APIs.

Frente a Shuffle: Si bien Shuffle es un SOAR de código abierto diseñado específicamente para ciberseguridad, tras las pruebas iniciales se consideró excesivamente rígido y limitado en comparación con las posibilidades de n8n.

Selección de n8n (n8n GmbH, s.f.): La decisión de implementar n8n se fundamentó en tres pilares:

- **Amplitud de integraciones nativas:** Su capacidad para conectar herramientas heterogéneas (desde Discord hasta bases de datos o servicios en la nube) recrea con fidelidad un entorno corporativo real donde la seguridad debe integrarse con el resto del ecosistema IT.
- **Modelo código abierto:** Al ser código abierto, ofrece un catálogo extenso de nodos comunitarios sin los costes prohibitivos de soluciones comerciales.
- **Modelo híbrido Low-Code + Code:** n8n permite diseñar flujos de forma visual pero admite la inserción de nodos de código (JavaScript o Python) cuando la lógica de tratamiento de datos es compleja. Esto lo hace más potente que las soluciones puramente visuales y más ordenado que el código puro.

Una decisión menor pero relevante fue la elección de Discord como canal de notificación e interfaz conversacional. Aunque en un entorno corporativo real lo habitual sería utilizar Microsoft Teams o Slack, se optó por Discord en el laboratorio por cuatro motivos: el conocimiento previo sobre la plataforma y sus posibilidades de personalización (hilos, embeds, formularios interactivos, roles y permisos granulares), sus webhooks bien documentados y sin coste asociado, la disponibilidad de un nodo nativo en n8n con soporte completo para todas estas funcionalidades, y la idoneidad para validar el patrón de interacción operador-bot que constituye una de las aportaciones de este trabajo. La sustitución por Teams o Slack en un despliegue productivo es directa, ya que n8n dispone de nodos equivalentes para ambas plataformas y la lógica de los flujos no se vería alterada.

Una decisión adicional, específica del flujo conversacional desarrollado, fue la elección del modelo de razonamiento que dirige al agente. Se evaluaron dos opciones: scripts deterministas con parseo manual y un agente IA con *tool calling*. Se optó por la segunda porque es la única que permite incorporar nuevas capacidades operativas sin tocar la lógica del agente —basta con añadir un nuevo subworkflow herramienta y describirlo textualmente— y la única que admite peticiones del operador en lenguaje natural sin atenerse a una sintaxis específica. Como modelo se eligió **qwen2.5:14b** servido localmente mediante **Ollama**. La elección se basó en dos criterios. El primero, empírico: durante la fase de pruebas se evaluaron varios modelos abiertos de tamaño comparable (entre ellos Llama 3 8B y Mistral 7B), y **qwen2.5:14b** fue el que ofreció un comportamiento más estable y consistente en tareas de tool calling y generación de salidas estructuradas en JSON, que son los dos requisitos críticos de los flujos desarrollados. El segundo, pragmático: la capacidad del hardware disponible en el laboratorio descarta modelos de mayor tamaño (32B o 70B), cuyos requisitos de memoria habrían hecho el flujo totalmente inviable. Esta decisión mantiene además el criterio de aislamiento aplicado al resto del laboratorio: la información sensible que circula por el agente (direcciones IP investigadas, fragmentos de alertas) no abandona el entorno controlado.

B. Descripción técnica de los flujos de trabajo

n8n se desplegó mediante Docker, garantizando su persistencia y facilidad de actualización. Sobre esta plataforma se han implementado dos flujos complementarios que comparten infraestructura y, parcialmente, subworkflows reutilizables.

Flujo 1 — Tratamiento automatizado de alertas SOAR. Aborda directamente el problema de la "fatiga de alertas" mediante una secuencia lógica de enriquecimiento y respuesta:

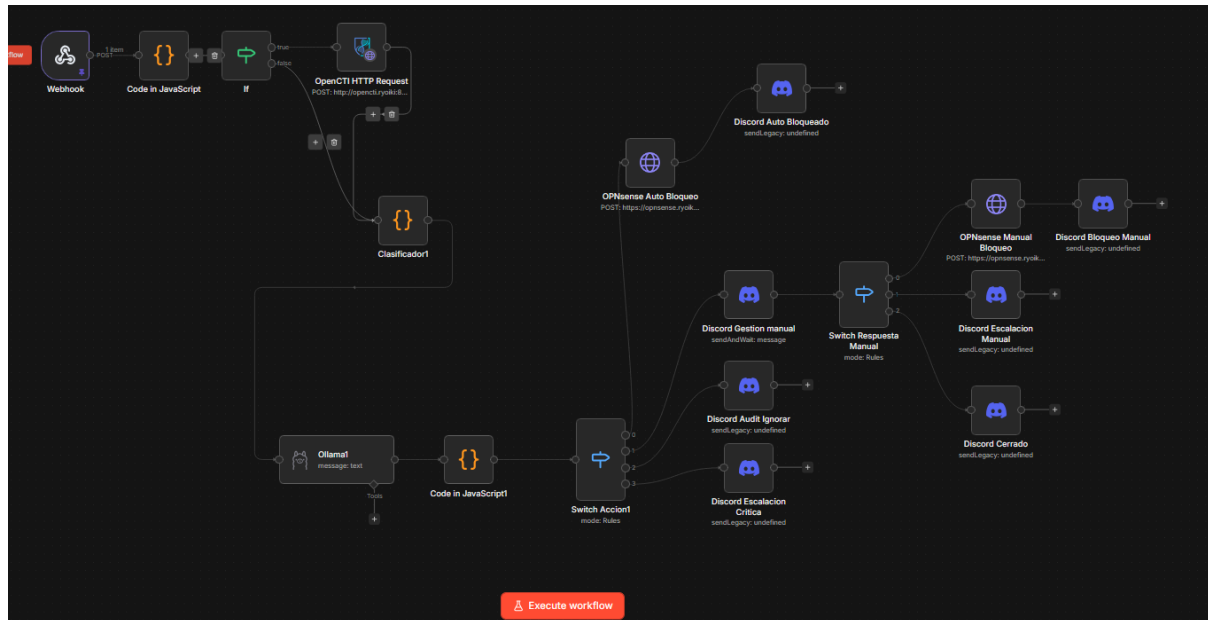


Figura 6: Workflow SOAR

- **Ingesta de alerta:** el flujo se activa mediante un webhook que recibe eventos críticos desde el manager de Wazuh.
- **Consulta de inteligencia (GraphQL):** se extraen los campos relevantes (como la dirección IP atacante) y se realiza una consulta a la API de OpenCTI. Para ello se empleó GraphQL, lo que permitió realizar peticiones precisas y obtener únicamente los objetos de inteligencia necesarios para el contexto.
- **Procesamiento mediante LLM:** los datos de la alerta y la inteligencia de OpenCTI se fusionan y se envían a un modelo de lenguaje mediante un prompt diseñado para la ocasión. El modelo actúa como un analista de Nivel 1: decide la severidad, estructura los datos y genera un resumen del incidente.
- **Ejecución de respuesta:** dependiendo del veredicto del flujo, el sistema desencadena una de cuatro acciones posibles: bloqueo automático en el firewall OPNsense, escalación al canal de Tier-2 en Discord, registro silencioso de auditoría, o consulta interactiva al analista mediante formulario en Discord.

Flujo 2 — Asistente conversacional con agente IA. Cubre el escenario complementario en el que el analista detecta una amenaza por inteligencia externa o por una investigación en curso, y necesita ejecutar una respuesta concreta sobre la infraestructura sin pasar por la consola de cada herramienta. El operador interactúa con el bot escribiendo **!bot <petición>** en un canal de Discord habilitado, y un agente IA decide en cada caso si responder conversacionalmente, invocar una herramienta operativa o solicitar aclaraciones.

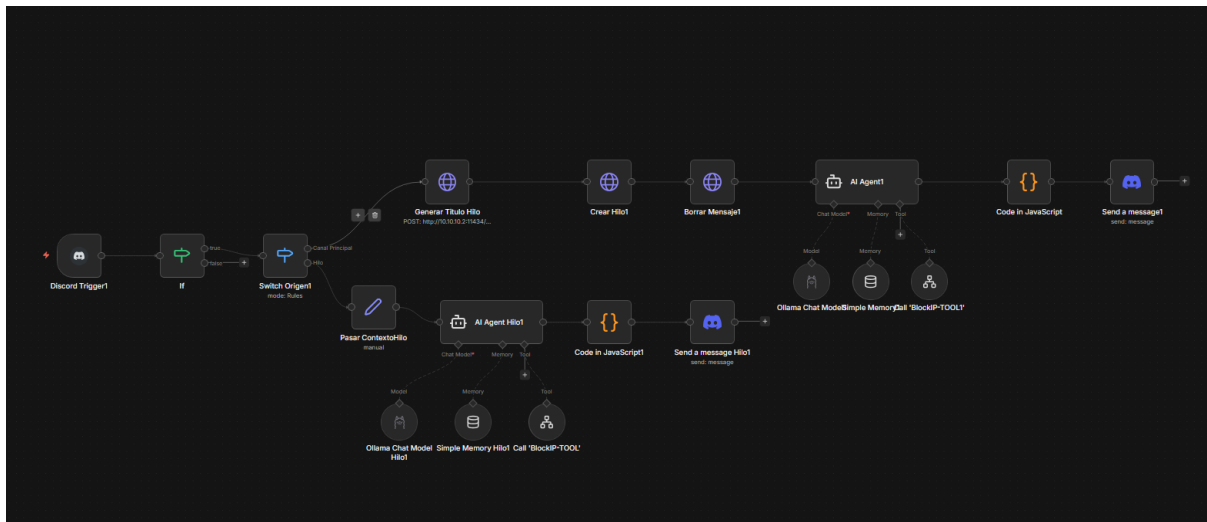


Figura 7: Workflow IA Agent

La arquitectura sigue el patrón "agente + catálogo de herramientas". El workflow principal aloja al agente conversacional, dotado de un modelo de lenguaje, una memoria simple segmentada por usuario y canal, y un conjunto de herramientas invocables. Cada herramienta es un subworkflow autocontenido que recibe parámetros, ejecuta la acción contra el sistema externo correspondiente y devuelve un resultado estructurado. Como primer caso de uso se ha implementado una herramienta de bloqueo de IPs (**BlockIP-TOOL**) que reutiliza el mismo subworkflow del flujo 1 para garantizar la coherencia entre ambos modos de operación: la acción real sobre OPNsense es idéntica, distinguiéndose únicamente el origen de la petición (manual o automático) para adaptar el formato de la notificación al canal de auditoría.

Conviene subrayar que la elección del bloqueo de IPs como caso de uso es estrictamente demostrativa. La aportación real del flujo conversacional no reside en la herramienta concreta, sino en el patrón arquitectónico que se establece: una infraestructura de agente capaz de incorporar nuevas capacidades operativas mediante la simple adición de subworkflows-herramienta, sin necesidad de modificar la lógica del agente ni el resto del flujo. Casos de uso naturales para futuras herramientas serían la consulta de IoCs en VirusTotal u OpenCTI, el desbloqueo de IPs como operación inversa, la creación de tickets en sistemas de gestión, el lanzamiento de búsquedas sobre Wazuh o la ejecución de comandos y recopilación de información en diferentes máquinas a la vez para aumentar la velocidad de auditorías.

La documentación detallada de ambos flujos se desarrolla en anexos independientes en formato Markdown. **ver anexos - (Workflow SOAR y BOT SOC discord)**

C. Desafíos técnicos y aprendizaje sobre la fragilidad de la automatización

La implementación de este componente supuso uno de los mayores retos de aprendizaje del TFG y aportó lecciones que trascienden los casos concretos.

En primer lugar, un cambio inesperado en la estructura de datos tras una actualización de la versión de Wazuh provocó la rotura total del flujo SOAR inicial. Esta incidencia obligó a rediseñar por completo la lógica de parseo y a profundizar en el uso de GraphQL, una tecnología sobre la que no se disponía de conocimiento previo. La necesidad de mapear esquemas de datos complejos bajo presión técnica sirvió para extraer una lección de ingeniería fundamental: las arquitecturas automatizadas son intrínsecamente frágiles ante cambios en terceros. En un entorno de producción real, este hallazgo justifica la necesidad de implementar estrategias de versionado y validación de esquemas para asegurar la continuidad del servicio de ciberinteligencia.

En segundo lugar, la implementación del flujo conversacional reveló dificultades específicas al integrar agentes IA en infraestructura operativa. Los modelos LLM cuantizados —los que pueden ejecutarse localmente en hardware modesto— presentan inconsistencias importantes en tareas de tool calling: durante las pruebas se observaron episodios en los que el modelo alucinaba confirmaciones afirmando haber bloqueado una IP sin haber invocado realmente la herramienta, verbalizaba su razonamiento interno en inglés mezclado con la respuesta final, y se confundía ante esquemas de parámetros amplios. La estabilización del comportamiento exigió combinar tres palancas: instrucciones imperativas en el system prompt contra la simulación de acciones, descripciones de herramienta con frases categóricas y ejemplos de disparo, y un nodo de saneamiento posterior que filtra la salida del modelo. La lección extraída es que un agente IA en producción no puede confiarse exclusivamente al modelo: requiere una envoltura defensiva que establezca su comportamiento.

Con la integración de n8n se completa el diseño e implementación técnica de la plataforma. El sistema no solo es capaz de conocer las amenazas y detectar ataques, sino de reaccionar ante ellos de forma orquestada y de habilitar al operador para conducir activamente la respuesta a través de lenguaje natural. Una vez descritos todos los componentes de la solución, el siguiente apartado abordará la validación práctica de este ecosistema mediante casos de uso que demuestren el cumplimiento de los objetivos planteados.

3.3 Recursos requeridos

Para la ejecución del proyecto se han utilizado los siguientes recursos, que se describen en detalle en la sección de Metodología y Herramientas empleadas. En este apartado se enumeran únicamente:

Recursos hardware

- Servidor dedicado *bare metal* contratado en OVHcloud
- Equipo portátil y de sobremesa para tareas de desarrollo, redacción y conexión remota al laboratorio

Recursos de infraestructura y virtualización

- Proxmox VE (hipervisor), Docker y Docker Compose (orquestración de contenedores)

Recursos de red y seguridad perimetral

- OPNsense (firewall y enrutador) / WireGuard (servidor VPN)

Plataforma de ciberinteligencia

- OpenCTI (plataforma CTI principal)
- Elasticsearch (almacenamiento) / Redis (caché) / RabbitMQ (broker de mensajería)
- Conectores OSINT (AlienVault OTX, MITRE ATT&CK, entre otros)

Detección y monitorización

- Wazuh Stack

Orquestación y automatización (SOAR)

- n8n (motor de automatización)
- Ollama como runtime de modelos de lenguaje locales
- Modelo qwen2.5:14b para razonamiento del agente IA

Notificación e interacción con el operador

- Discord como canal de alertas e interfaz conversacional del bot SOC

Recursos de desarrollo y documentación

- Obsidian (redacción y gestión de la documentación técnica en formato Markdown)
- GitHub (repositorio privado para versionado de las notas de Obsidian y anexos del proyecto)
- Editor de texto / IDE para edición de archivos de configuración y código

Recursos humanos

- Dirección y supervisión del proyecto a cargo del tutor del TFG, D. Ignacio Cea Fornies

3.4 Presupuesto

Este apartado presenta la evaluación económica del proyecto siguiendo dos perspectivas complementarias: el coste real del laboratorio donde se ha desarrollado el TFG y el coste estimado de la misma solución desplegada en una organización productiva. Antes de presentar las cifras, conviene aclarar el porqué de este enfoque dual.

3.4.1 ¿Por qué se elaboran dos presupuestos?

El proyecto produce un sistema funcional que puede ser evaluado desde dos contextos económicos distintos:

- **El coste de desarrollo:** lo que ha supuesto realmente construir y validar la solución en un entorno de laboratorio académico, con hardware homelab y software de código abierto versión comunitaria.
- **El coste de despliegue productivo:** lo que costaría implantar y operar esa misma solución en una organización real, donde aparecen requisitos de disponibilidad, soporte, compliance y personal cualificado que el laboratorio no tiene.

Presentar únicamente el primero subestima el valor económico real del trabajo y oculta los costes ocultos del código abierto en producción. Presentar únicamente el segundo desliga el presupuesto del esfuerzo real invertido en el TFG.

3.4.2 ¿Por qué el coste final depende de la infraestructura de implantación?

Una observación crítica para interpretar correctamente las cifras: el mismo software, instalado y configurado de forma equivalente, puede tener un coste anual de 0 € o de decenas de miles de euros según el contexto operativo en el que se despliegue. Esta diferencia no proviene del software en sí — el código fuente es el mismo en la edición Community y en la Enterprise — sino del entorno que lo rodea:

- **Escala y redundancia:** el laboratorio funciona sobre un único nodo Proxmox sin alta disponibilidad. Una empresa exige cluster con HA, sitio de DR (recuperación ante desastres), balanceadores y firewalls en pareja, lo que multiplica el coste de hardware y, por extensión, el de licencias (que se facturan por socket o por instancia).
- **Acuerdos de nivel de servicio (SLA):** el laboratorio tolera caídas y actualizaciones inestables; un entorno productivo necesita repositorios estables y soporte con tiempos de respuesta garantizados (las suscripciones Premium de Proxmox y Wazuh, o el soporte Business de OPNsense, no aportan funcionalidad adicional sino fiabilidad operativa).
- **Cumplimiento normativo:** una empresa sometida al ENS, ISO 27001 o NIS2 necesita audit logs, RBAC (Control de Acceso Basado en Roles) granular, SSO (Inicio de Sesión Único) empresarial y trazabilidad — funcionalidades que precisamente diferencian las ediciones Enterprise (OpenCTI EE, n8n Enterprise) de ediciones Community.
- **Personal y operación:** el laboratorio lo opera una persona en horas de TFG; un Centro de Operaciones de Seguridad (SOC) corporativo necesita analistas de varios niveles cubriendo turnos extendidos. Este coste recurrente (OPEX de personal) es habitualmente el mayor del presupuesto enterprise, muy por encima de cualquier licencia.
- **Volumen de datos:** la facturación de plataformas como Wazuh Cloud depende del volumen de logs ingeridos y del número de endpoints monitorizados. Pasar de 5 VMs a 500 endpoints no multiplica el coste por 100 de forma lineal, pero tampoco se mantiene constante.

Por todo lo anterior, las cifras del presupuesto enterprise deben leerse como una estimación razonada para el escenario concreto descrito, no como un precio universal del software. Cualquier cambio en el tamaño de la organización, el sector, el nivel de compliance exigido o la cobertura horaria del SOC modificaría sustancialmente el resultado final.

3.4.3 Mapeo del stack: laboratorio frente a edición enterprise

Para que la comparación de costes sea homogénea, en el presupuesto de la empresa simulada se ha sustituido cada componente del stack del laboratorio por su versión Enterprise oficial del mismo producto, en lugar de por una herramienta comercial competidora. Esto permite aislar el coste atribuible al cambio de contexto operativo (laboratorio frente a producción), separándolo del coste atribuible a un cambio de ecosistema tecnológico.

Stack utilizado en el laboratorio (Community)	Versión enterprise equivalente del mismo producto
Proxmox VE Community	Proxmox VE Premium (suscripción por socket)
OPNsense Community Edition	OPNsense Business Edition + Business Support + Proofpoint ET Pro Ruleset
Wazuh (código abierto)	Wazuh Cloud / soporte Enterprise vía partner
OpenCTI Community Edition	OpenCTI Enterprise Edition (Filigran)
n8n Community	n8n Enterprise (self-hosted con licencia comercial)

Tabla 1: Mapeo Stack

3.4.4 Presupuesto 1 — Laboratorio (TFG)

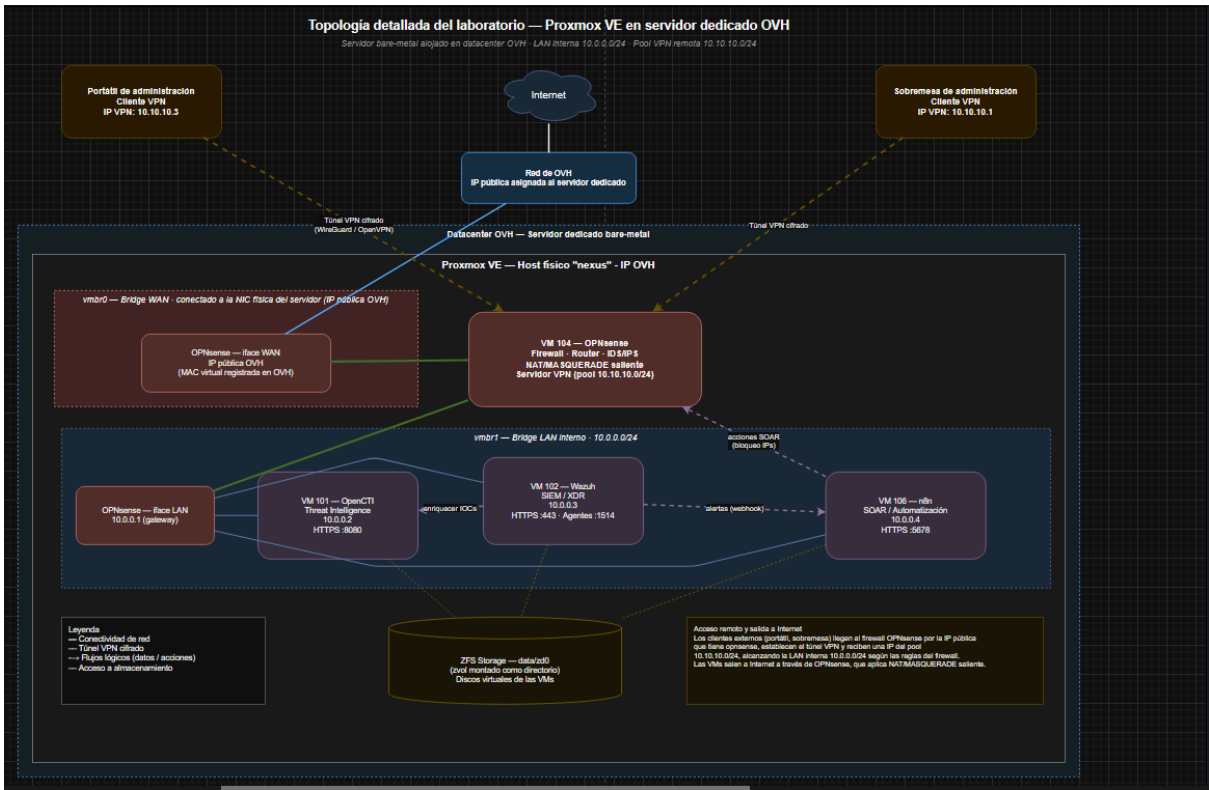


Figura 8: Red Lab

Recoge el coste real del entorno de desarrollo y validación del proyecto. El equipo técnico se valora a precio de mercado de adquisición nueva, aunque parte del mismo ya estaba disponible. El software es íntegramente código abierto en su edición community, con coste de licencia nulo.

Tipo de coste	Valor	Comentarios
Horas de trabajo en el proyecto	4 875 €	Estudiante TFG: 325 h × 15 €/h
Horas de trabajo en el proyecto	1 400 €	Tutor TFG: ~20 h × 70 €/h (orientación y revisiones)
Equipo técnico utilizado	200 €	Servidor bare metal
Equipo técnico utilizado	5 000 €	Portátil y sobremesa
Software	0 €	Proxmox VE Community
Software	0 €	OPNsense Community
Software	0 €	Wazuh

Tipo de coste	Valor	Comentarios
Software	0 €	OpenCTI Community Edition
Software	0 €	n8n Community
Software	0 €	Linux (Debian) y utilidades (Docker, etc.)
Estudios e informes	0 €	Documentación oficial de cada producto y blogs técnicos (recursos gratuitos)
TOTAL LABORATORIO	≈ 11 475 €	De los cuales solo ~5 200 € son desembolso real (CAPEX); el resto es valoración del tiempo

Tabla 2: presupuesto lab

3.4.5 Presupuesto 2 - Empresa simulada (mediana, ~500 empleados)

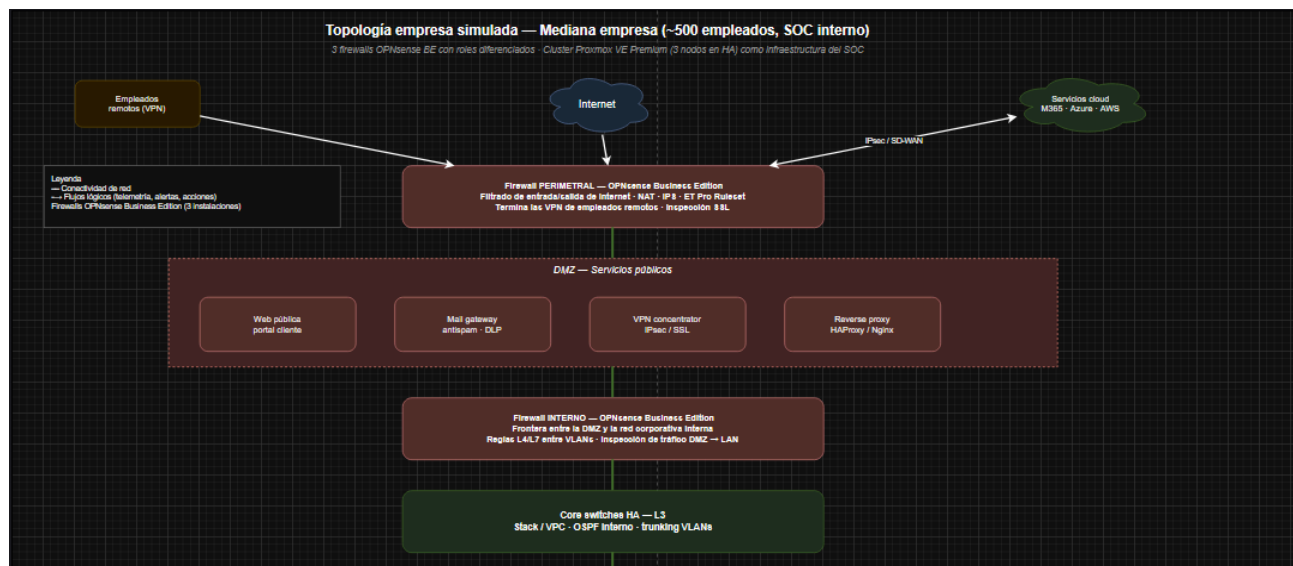


Figura 9: Red empresa 1

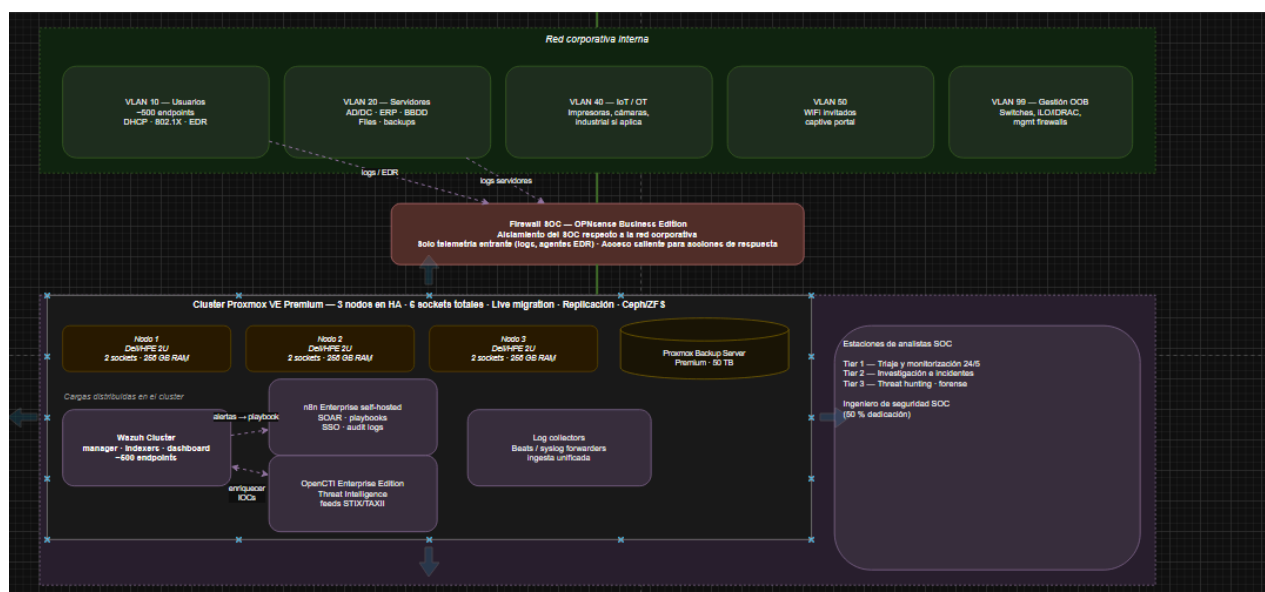


Figura 10: Red empresa 2

Estimación del coste de implantación y primer año de operación de la solución desarrollada en una empresa de tamaño medio con SOC interno. Se diferencia el CAPEX (inversión inicial: hardware, despliegue) del OPEX (gasto anual recurrente: personal, licencias, servicios). El software es el mismo del laboratorio, en su edición Enterprise.

El dimensionamiento parte de un cluster Proxmox de tres nodos, configuración que constituye el mínimo profesional para garantizar alta disponibilidad: un número impar de nodos permite mantener el funcionamiento del cluster ante la caída de uno de ellos, evitando situaciones de split-brain que se producirían con solo dos nodos. Esta decisión se refleja en el coste de hardware (tres servidores físicos) y en las licencias de infraestructura (Proxmox VE, OPNsense y su ruleset Proofpoint ET Pro), cuyo modelo de licenciamiento por nodo o por instancia obliga a multiplicar el coste unitario por

tres. En cambio, las plataformas de aplicación (Wazuh Cloud, OpenCTI Enterprise y n8n Business) se licencian a nivel de organización o de instancia lógica, por lo que no se replican aunque por debajo se ejecuten distribuidas en el cluster.

Coste anual de software (versión Enterprise del stack del laboratorio):

Tipo de coste	Valor	Comentarios
Software	6 600 €/año	3 x Proxmox VE Premium Subscription 2 CPUs/year - 2200 € x 3
Software	447 €/año	OPNsense Business Edition: 149 €/año x 3 instalaciones
Software	987 €/año	OPNsense Business Support: 329 €/año x 3
Software	2 247 €/año	Proofpoint ET Pro Ruleset: 749 €/año x 3 firewalls
Software	18 000 €/año	Wazuh Cloud / soporte enterprise para ~500 endpoints (estimación basada en referencias públicas) ~3€/endpoint x 500 x 12 meses
Software	25 000 €/año	OpenCTI Enterprise Edition (Filigran): licencia + soporte (pricing privado, estimación aproximada basada en referencias públicas de referencias públicas del mercado CTI (≈ 85.000 €))
Software	52 452 €/año	n8n business self-hosted < 100 personas: licencia, audit logs y soporte (estimación, ya que el precio enterprise es bajo demanda personalizada)
SUBTOTAL SOFTWARE	≈ 105 733 €/año	

Tabla 3: presupuesto empresa software

Tipo de coste	Valor	Comentarios
IMPLANTACIÓN		
Horas de trabajo en el proyecto	15 000 €	Arquitecto SOC: 200h x 75 €/h. Diseño, casos de uso, runbooks
Horas de trabajo en el proyecto	60 000 €	2 ingenieros senior x 500 h x 60 €/h. Despliegue NGFW (Firewall de nueva generación), SIEM, EDR
Horas de trabajo en el proyecto	30 000 €	2 ingenieros junior x 500 h x 30 €/h. Configuración, agentes, tuning
Horas de trabajo en el proyecto	10 000 €	Project Manager: 200 h x 50 €/h. Coordinación y reporting
Equipo técnico	12 000 €	3 firewalls hardware (OPNsense DEC4240)
Equipo técnico	6 600 €	3 x Core switches HA (Aruba / Cisco)

Tipo de coste	Valor	Comentarios
Equipo técnico	15 000 €	Almacenamiento adicional para SIEM y CTI logs
Equipo técnico	15 000 €	Racks, SAI, KVM, cableado estructurado
Equipo técnico	45 000 €	3 servidores físicos del cluster Proxmox
CAPEX inicial	≈ 208 600 €	
OPERACIÓN ANUAL		
Horas de trabajo en el proyecto	180 000 €/año	3 analistas SOC (mix T1/T2/T3, fully loaded a 60 mil€/año, media entre max y min)
Horas de trabajo en el proyecto	40 000 €/año	1 ingeniero seguridad (50 % tiempo 40k, fully loaded ~80k€/año)
Estudios e informes	25 000 €/año	Pentest anual externo
Estudios e informes	12 000 €/año	Auditoría ENS (Esquema nacional de seguridad) / ISO 27001
Estudios e informes	5 000 €/año	Suscripción a feeds CTI premium adicionales, variación en función del feed. Q-FEED aproximadamente 4 000-5 000 €
OPEX anual	≈ 262 000 €/año	
TOTAL EMPRESA — AÑO 1	≈ 576 333 €	Software anual, Capex+Opex

Tabla 4: presupuesto empresa Capex & Opex & Software

3.5 Viabilidad

El análisis de viabilidad de un proyecto de ciberinteligencia presenta una particularidad respecto a otros desarrollos de ingeniería: su retorno no se mide en ingresos generados, sino en pérdidas evitadas. Por ello, este apartado evalúa, primero, la relación coste-beneficio de la solución y, después, su sostenibilidad técnica a futuro.

3.5.1 Viabilidad económica

Las cifras relevantes a efectos de viabilidad son las del despliegue corporativo simulado, que sitúa el coste del primer año en torno a 576.000 €, con una inversión inicial (CAPEX) próxima a 208.600 € y un gasto operativo anual cercano a 367.700 € que combina los 262.000 € de OPEX recurrente (Tabla 4) y los 105.733 € de software *Enterprise* anual (Tabla 3).

Para juzgar si esta cifra es asumible, conviene contrastar con el coste de no implantar la solución. Según el informe *Cost of a Data Breach Report 2024* de IBM Security, ya citado en el estado del arte, el coste medio global de una única brecha de datos asciende a 4,88 millones de dólares (IBM Security, 2024). En la edición de 2025 de este mismo informe la cifra se ajusta ligeramente a la baja, hasta los 4,44 millones de dólares (IBM Security, 2025), confirmando un orden de magnitud estable. En este contexto, y asumiendo paridad aproximada euro-dólar ($1\text{ €} \approx 1,17\text{ \$}$ en el momento de este trabajo) a efectos comparativos dada la similitud de magnitud entre ambas divisas, la inversión anual estimada representa aproximadamente un 11 % del coste medio de un único incidente, por lo que basta con que la plataforma contribuya a evitar o contener un único incidente significativo a lo largo de su vida útil para que la inversión quede justificada económicamente.

A esta justificación directa se suman dos eficiencias adicionales: la reducción del tiempo de triaje mediante el enriquecimiento automatizado y la disminución de la ventana de exposición gracias a la orquestación con n8n. Ambas mejoras inciden sobre las dos variables que más encarecen una brecha según el informe IBM: el tiempo de detección y el tiempo de contención.

3.5.2 Sostenibilidad a futuro

La sostenibilidad del resultado depende tanto de la evolución del *stack* como de las dependencias críticas que podrían comprometer su continuidad operativa.

En el plano positivo, los componentes principales (Proxmox VE, OpenCTI, Wazuh, OPNsense y n8n) cuentan con comunidades activas y periodos de actualización frecuentes. Los estándares sobre los que se apoya la plataforma están mantenidos por organizaciones de referencia internacional como OASIS, lo que minimiza el riesgo de quedar atado a un formato de datos abandonado. Adicionalmente, el carácter contenedorizado del despliegue facilita migraciones entre versiones y la portabilidad del entorno completo a otra infraestructura física.

No obstante, deben reconocerse dos inconvenientes. La primera es la fragilidad de las integraciones automatizadas frente a cambios en terceros, evidenciada durante el desarrollo cuando una actualización de Wazuh modificó el esquema de datos y rompió el flujo SOAR; esta lección obliga a incorporar prácticas de validación de esquemas y pruebas de regresión antes de aplicar

actualizaciones en producción. La segunda es el factor humano: la plataforma libera al analista de tareas mecánicas, pero exige perfiles capaces de mantener conectores, ajustar reglas de correlación y supervisar agentes de IA.

En conjunto, la solución presenta una sostenibilidad razonable, condicionada a una operación profesional que asuma el mantenimiento continuo como parte del servicio. La arquitectura modular facilita esta evolución, dado que cualquiera de los componentes puede sustituirse o actualizarse de forma independiente sin necesidad de rediseñar el conjunto.

3.5.3 Contribución a la sostenibilidad

El proyecto contribuye a varios Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de Naciones Unidas:

- **ODS 9 (Industria, innovación e infraestructura):** el proyecto aporta una infraestructura corporativa de ciberinteligencia construida íntegramente con software libre, demostrando que la innovación en seguridad no requiere necesariamente soluciones propietarias y democratizando el acceso a capacidades CTI avanzadas para organizaciones con recursos limitados.
- **ODS 12 (Producción y consumo responsables):** la decisión arquitectónica de priorizar AlienVault OTX como fuente principal y limitar el número de conectores activos responde a un criterio de uso eficiente de recursos computacionales, evitando el sobredimensionamiento típico de plataformas empresariales que consumen energía innecesaria en procesar volúmenes de datos no aprovechados. El despliegue contenerizado mediante Docker permite además una utilización óptima del hardware, reduciendo la huella energética frente a despliegues sobre máquinas físicas dedicadas.
- **ODS 16 (Paz, justicia e instituciones sólidas):** el proyecto refuerza la soberanía digital europea al ofrecer una alternativa de ciberinteligencia no sujeta a legislaciones extraterritoriales como el CLOUD Act estadounidense, contribuyendo a la autonomía estratégica de las organizaciones europeas en materia de seguridad.

Adicionalmente, en su dimensión social, la arquitectura propuesta reduce la fatiga de alertas y la carga cognitiva sobre los analistas SOC mediante automatización y enriquecimiento contextual, mejorando las condiciones laborales del personal técnico y la calidad de la respuesta ante incidentes.

3.6 Resultados del proyecto

A continuación se exponen los resultados obtenidos al término del proyecto, manteniendo la trazabilidad con los objetivos específicos definidos en el capítulo 2. Posteriormente se detalla el plan de pruebas que valida el funcionamiento del sistema y, por último, se recogen las desviaciones identificadas respecto a la planificación inicial.

3.6.1 Resultados conforme a los objetivos específicos

Objetivo 1 — Despliegue de la infraestructura virtualizada soberana. El primer objetivo se ha cumplido íntegramente. Sobre el servidor *bare metal* contratado en OVHcloud se desplegó Proxmox VE como hipervisor, alojando el conjunto de máquinas virtuales que componen el laboratorio en el segmento de red 10.0.0.0/24. La asignación final de recursos (10 vCores y 40 GB de RAM para OpenCTI, 4 vCores y 8 GB para Wazuh, y configuraciones más ligeras para OPNsense y n8n) permitió sostener la carga de las cuatro VMs en producción simultánea sin degradación apreciable del rendimiento. La configuración de red base, incluyendo el enrutador inicial y la posterior transición al firewall corporativo, se documenta de forma detallada en los anexos correspondientes (**ver Anexo - Configuración VM Router y Anexo - OPNsense**). El control total sobre la red, la jurisdicción europea del proveedor y la ausencia de dependencias con nubes extraterritoriales validan el principio de soberanía planteado.

Objetivo 2 — Implementación y optimización de OpenCTI. El núcleo de la plataforma quedó plenamente operativo en su versión **6.9.22**, con sus dependencias contenerizadas (Elasticsearch, Redis, MinIO, RabbitMQ y tres réplicas del *worker*) gestionadas mediante Docker Compose (**ver Anexo - OpenCTI**). La operación cotidiana del *stack* se simplificó mediante el *script* de administración personalizado descrito en el **Anexo - opencti-control**. Se desarrollaron los tres conectores propios contemplados en los objetivos, dos de los cuales están plenamente operativos: el conector *Multi-Feed* opera de forma estable y nutre la base de datos sin requerir claves API ni autenticación contra ningún servicio externo (**ver Anexo - multifeed connector documentación**); el conector de enriquecimiento basado en Gemini se completó técnicamente y cumple su función de generar notas en *Markdown* sobre los objetos STIX, aunque su uso queda limitado por las restricciones de la capa gratuita de la API (**ver Anexo - gemini connector documentación**); y el conector de *stream*, aunque diseñado, no se desplegó en producción al no existir un sistema consumidor que justificara el flujo en tiempo real. Junto a estos desarrollos propios, el conector oficial de AlienVault OTX se mantuvo como fuente principal de inteligencia tras la fase de optimización, en línea con la estrategia de uso sostenible de recursos descrita en los objetivos.

Objetivo 3 — Integración de los sistemas de detección y seguridad perimetral. Wazuh quedó operativo como nodo central SIEM/XDR en su arquitectura *all-in-one* nativa sobre Debian, con agentes desplegados en todas las máquinas críticas del laboratorio y comunicaciones cifradas mediante TLS (**ver Anexo - Wazuh**). OPNsense, por su parte, asumió las funciones de *gateway* del segmento corporativo, con WireGuard configurado para el acceso administrativo remoto, Unbound DNS para la resolución interna del dominio **.ryoiki** y un *alias* dinámico (**Bloqueo_SOAR**) alimentado vía API por la capa de orquestación con las IPs confirmadas como maliciosas tras el análisis automatizado (**ver Anexo - OPNsense**).

Objetivo 4 — Desarrollo de los flujos de orquestación y automatización. La plataforma n8n se desplegó mediante Docker Compose siguiendo el procedimiento documentado en el **Anexo - n8n**, y sobre ella se construyeron los dos flujos previstos. El primero implementa un SOAR completo que recibe alertas de Wazuh por *webhook*, las enriquece con consultas GraphQL a OpenCTI, las analiza mediante un modelo de lenguaje local (**qwen2.5:14b** servido por Ollama) y desencadena automáticamente una de cuatro acciones posibles: bloqueo en OPNsense, escalación a Tier-2,

registro de auditoría o consulta interactiva al analista (**ver Anexo - Workflow SOAR**). El segundo flujo implementa un asistente conversacional desplegado en Discord que, mediante el patrón "agente + catálogo de herramientas", permite al analista ejecutar acciones operativas sobre la infraestructura mediante lenguaje natural (**ver Anexo - Bot SOC Discord**). Ambos flujos comparten infraestructura y, en el caso del bloqueo de IPs, comparten también el subworkflow operativo, garantizando coherencia entre el modo automático y el manual.

Objetivo 5 — Validación mediante casos de uso simulados. Este último objetivo se desarrolla en detalle en el siguiente apartado, ya que constituye el plan de pruebas del proyecto.

3.6.2 Plan de pruebas y casos de uso validados

La validación del sistema se ha realizado mediante un total de once casos de uso ejecutados en el laboratorio, distribuidos entre los dos flujos de orquestación. La descripción exhaustiva de cada escenario, incluyendo los comandos de simulación, las capturas de cada etapa del recorrido y las trazas de ejecución, se recoge en los anexos correspondientes a cada flujo.

Flujo SOAR automatizado (seis casos). Se diseñaron seis escenarios para cubrir las ramas decisionales del flujo, todos ellos documentados al detalle en el **Anexo - Workflow SOAR**:

- **Caso 1 — Ataque de fuerza bruta SSH desde una IP previamente fichada en OpenCTI.** Validó la rama de bloqueo automático: la IP atacante se añadió al *alias Bloqueo_SOAR* de OPNsense y se publicó la notificación correspondiente en Discord sin intervención humana.
- **Caso 2 — Modificación de un fichero PHP con código de webshell monitorizado por FIM (Monitoreo de integridad de archivos).** Validó la rama de escalación crítica con notificación *@here* al canal de Tier-2.
- **Caso 3 — Inicio de sesión SSH legítimo.** Validó la rama de auditoría silenciosa, demostrando que el sistema discrimina correctamente el tráfico benigno.
- **Caso 4 — Ataque de fuerza bruta desde una IP sin contexto en OpenCTI.** Validó la rama de consulta al analista mediante formulario interactivo en Discord, ejecutándose el bloqueo solo tras la decisión humana.
- **Caso 5 — Fuerza bruta desde una IP interna.** Validó el comportamiento defensivo del sistema, que escala en lugar de bloquear cuando la acción podría afectar a infraestructura propia.
- **Caso 6 — Modificación rutinaria sin contexto malicioso.** Validó el cierre como falso positivo tras consulta al analista.

Flujo conversacional (cinco casos). El bot SOC desplegado en Discord se validó mediante cinco escenarios documentados en el **Anexo - Bot SOC Discord**:

- **Casos 1 y 2 — Bloqueo de IPs invocado mediante lenguaje natural.** Validaron la ejecución correcta de la herramienta operativa, comprobando además que el sistema crea automáticamente un hilo nuevo cuando la petición proviene del canal principal y mantiene la conversación en el mismo hilo cuando ya existe.

- **Caso 3 — Petición conversacional sin acción asociada.** Validó la capacidad del agente para responder en lenguaje natural sin invocar herramientas cuando la petición no requiere acción operativa.
- **Caso 4 — Petición incompleta sin parámetros suficientes.** Validó la regla anti-alucinación del *system prompt*: ante una petición incompleta ("bloquea esa IP" sin especificar cuál), el agente solicita aclaración en lugar de inventar un valor plausible, lo que previene un fallo crítico de seguridad.
- **Caso 5 — Investigaciones paralelas en hilos distintos.** Validó el aislamiento de la memoria conversacional por hilo, demostrando que dos investigaciones simultáneas del mismo operador no contaminan sus contextos respectivos.

Volumen de inteligencia gestionado. Tras el periodo de operación del laboratorio con los conectores activos, la plataforma alcanzó un volumen de aproximadamente 180.000 indicadores de compromiso, 1.570 muestras de malware catalogadas, 464 informes de inteligencia y 325 conjuntos de intrusión (intrusion sets) asociados a actores de amenaza conocidos. El motor de grafo de OpenCTI generó además más de 300.000 relaciones entre estos objetos, materializando el modelo STIX 2.1 sobre el que se sustenta la plataforma. Este volumen, documentado en la [Figura 3](#) del apartado correspondiente a este componente, demuestra que la arquitectura desplegada no opera con un conjunto reducido de datos de prueba, sino con el flujo real de inteligencia generado por las fuentes integradas, lo que valida su capacidad operativa bajo condiciones próximas a un entorno productivo.

Resultados temporales del flujo automático. En las pruebas realizadas en el laboratorio, el tiempo total transcurrido entre la generación de la alerta por Wazuh y la efectividad del bloqueo en OPNsense se sitúa en el entorno de los tres minutos. Esta cifra debe interpretarse como un orden de magnitud propio del hardware del laboratorio y no como una métrica formal de rendimiento, dado que el principal componente de latencia es la inferencia del modelo de lenguaje local sobre la GPU disponible. En un entorno corporativo con aceleración dedicada o con un modelo *cloud* sustitutivo, la latencia esperable se reduciría sensiblemente.

3.6.3 Desviaciones respecto a los objetivos iniciales

El desarrollo del proyecto produjo varias desviaciones respecto a la planificación inicial, todas ellas justificadas y sin impacto sobre el cumplimiento del objetivo general.

- La primera y más relevante se refiere a la estrategia de ingesta de inteligencia. La planificación inicial contemplaba el despliegue simultáneo de un mayor número de conectores de fuentes externas para nutrir la plataforma. Tras observar el consumo real de Elasticsearch durante la fase de pruebas, se decidió reducir el conjunto activo a AlienVault OTX como fuente principal junto a los conectores propios, descartando el resto para evitar saturar la base de datos del laboratorio. Esta decisión se alinea con el principio de uso sostenible de recursos planteado en los objetivos.

- La segunda desviación afecta al conector de enriquecimiento basado en Gemini. Aunque su desarrollo concluyó satisfactoriamente y la integración con la API de OpenCTI funciona correctamente, las restricciones de cuota de la capa gratuita de la *Google AI for Developers API* impiden su uso operativo de manera continua en el laboratorio. Esta limitación, así como el análisis técnico que justifica el uso futuro de cuotas empresariales, se documenta en el **Anexo - gemini connector documentación**.
- La tercera desviación corresponde al conector de *stream*. Aunque su estructura base se diseñó conforme a lo planificado, se decidió no desplegarlo en la arquitectura final al no existir un sistema consumidor intermedio que requiriera un consumo persistente de datos en bruto.
- La cuarta desviación afectó al despliegue de Wazuh. La planificación inicial contemplaba su instalación contenerizada vía Docker, en coherencia con el resto del *stack*, pero los conflictos detectados en la gestión de la PKI interna y los volúmenes persistentes obligaron a migrar a una instalación nativa sobre el sistema operativo. El procedimiento finalmente adoptado se recoge en el **Anexo - Wazuh**. Este cambio aportó la estabilidad requerida sin afectar a la funcionalidad prevista.
- Por último, durante la fase de configuración de DNS interno se identificó que el dominio **.nexus** inicialmente previsto para el laboratorio formaba parte de la lista de precarga HSTS de los navegadores modernos, lo que impedía el acceso a los servicios mediante HTTP o certificados autofirmados. Este hallazgo obligó a sustituirlo por un dominio inventado (**.ryoiki**), sin colisión con ningún TLD (Dominio de nivel superior) real ni políticas HSTS aplicables, según la configuración detallada en el **Anexo - OPNsense**.

En conjunto, las desviaciones identificadas no comprometen el cumplimiento del objetivo general del proyecto y, en varios casos, aportaron lecciones de ingeniería sobre la fragilidad de las integraciones automatizadas y la importancia del diseño defensivo, lecciones que se desarrollan con mayor profundidad en el capítulo de discusión.

Capítulo 4. DISCUSIÓN

Una vez expuestos los resultados, este capítulo interpreta críticamente lo construido en tres planos: limitaciones, metodología e impacto.

4.1 Limitaciones de la solución y de la tecnología empleada

La principal limitación afecta al razonamiento automatizado del flujo SOAR. El uso de un modelo cuantizado servido localmente respondía al principio de soberanía del dato, pero arrastra la fiabilidad propia de los modelos de tamaño moderado en tareas de *tool calling*. La envoltura defensiva diseñada estabiliza el comportamiento, pero no elimina el problema de raíz. En un entorno corporativo esta limitación es perfectamente abordable: basta con sustituir el modelo local por uno mayor desplegado on-premise con hardware dedicado, conservando intacto el principio de soberanía. La modularidad de la arquitectura permite este cambio sin rediseño.

La segunda limitación es la fragilidad de las integraciones automatizadas frente a cambios en APIs de terceros, evidenciada durante el desarrollo. Es un fenómeno general de cualquier arquitectura SOAR, no una debilidad específica del proyecto, y su mitigación en producción exige validación de esquemas y pruebas de regresión que un entorno académico no puede sostener.

4.2 Reflexión sobre la metodología empleada

La metodología iterativa por componentes resultó adecuada y demostró su valor durante la integración. No obstante, dejó al descubierto un punto ciego: los problemas que solo emergen en la conjunción de componentes escapan al enfoque modular. La lección es clara: el aislamiento por componentes resuelve la complejidad del despliegue, pero no la de la integración, que requiere un entorno de pruebas separado del operativo.

4.3 Impacto y aportación del proyecto

El proyecto deja dos aportaciones que exceden el ámbito académico. La primera es la demostración de que es viable construir una plataforma corporativa de ciberinteligencia íntegramente con software libre, una vía técnica relevante tanto para organizaciones medianas excluidas del mercado CTI por coste como para grandes corporaciones que prioricen soberanía técnica, auditabilidad y alineamiento con marcos regulatorios europeos como la Directiva NIS 2 —que obliga a sectores críticos a reforzar sus capacidades de detección y respuesta— o el Esquema Nacional de Seguridad (ENS). La segunda es arquitectónica: la implementación del patrón "agente conversacional + catálogo de herramientas" como interfaz operativa sobre infraestructura de seguridad. Esta aportación constituye un componente innovador del proyecto, ya que el uso de agentes IA como capa de operación sobre sistemas SOC es una tendencia emergente en el sector y no existen, hasta donde se ha podido verificar, soluciones de código abierto que integren este patrón con plataformas CTI consolidadas como OpenCTI. El diseño es además extensible, ya que admite la incorporación de nuevas capacidades operativas mediante la simple adición de subworkflows-herramienta.

Capítulo 5. CONCLUSIONES

5.1 Conclusiones del trabajo

El objetivo general planteado al inicio del proyecto consistía en diseñar, implementar y validar un prototipo funcional de una plataforma corporativa de ciberinteligencia *on-premise* basada en OpenCTI, capaz de centralizar la gestión de amenazas y orquestar automáticamente la distribución de inteligencia hacia los sistemas defensivos. A la luz de los resultados expuestos en el capítulo 3, este objetivo se ha cumplido de forma íntegra.

La plataforma desplegada integra cinco pilares (Proxmox VE, OpenCTI, Wazuh, OPNsense y n8n) operando de manera coordinada sobre un entorno virtualizado bajo control directo. La inteligencia ingerida desde fuentes externas y desde los conectores propios desarrollados se almacena, se enriquece y se distribuye automáticamente hacia los sistemas defensivos, donde alimenta tanto la detección (Wazuh) como la respuesta perimetral (OPNsense). La capa de orquestación implementada en n8n cierra el ciclo de inteligencia mediante dos flujos complementarios, uno automatizado y otro conversacional, ambos validados con un total de once casos de uso que cubren las distintas ramas decisionales del sistema.

Más allá del cumplimiento funcional, el proyecto demuestra que es viable construir una plataforma corporativa de ciberinteligencia íntegramente con software libre, sin renunciar a las capacidades operativas características de las soluciones comerciales del mercado y respetando los principios de soberanía del dato y cumplimiento normativo planteados en la justificación inicial.

5.2 Conclusiones personales

La elección de este tema responde a un interés genuino por la ciberseguridad y a la voluntad de dedicarme profesionalmente a este sector tras finalizar el grado. El proyecto ha confirmado esa orientación y me ha permitido construir, sobre un caso real, una visión integral del funcionamiento de los servicios CTI y SOC, áreas en las que pretendo desarrollar mi carrera.

A nivel personal, el reto más exigente del proyecto no fue técnico sino emocional. La rotura del flujo SOAR tras una actualización de Wazuh, ya analizada desde el punto de vista metodológico en el capítulo de discusión, resultó especialmente frustrante también en el plano personal, ya que el flujo había funcionado correctamente con anterioridad y los casos de uso de validación todavía no se habían ejecutado, lo que implicó un trabajo de reconstrucción importante en una fase avanzada del proyecto. A esta dificultad se sumó la gestión del propio ritmo de trabajo: compatibilizar el TFG con la vida personal a lo largo de varios meses ha exigido una disciplina que no se aprende en ninguna asignatura del grado y que constituye, en sí misma, uno de los aprendizajes más valiosos del proceso.

En el plano académico, sí me gustaría señalar una observación concreta. El desarrollo del flujo SOAR exigió un primer contacto con GraphQL para construir las consultas necesarias a la API de OpenCTI, lo que supuso una curva de aprendizaje considerable al partir sin conocimientos previos de este lenguaje. Una mayor exposición durante el grado a paradigmas de consulta alternativos al SQL tradicional habría facilitado este aprendizaje y resulta especialmente relevante en el ámbito de la ciberseguridad, donde plataformas como OpenCTI se apoyan en estos modelos.

Por último, el trabajo realizado no se concibe como un ejercicio académico cerrado. La intención es trasladar la solución a un entorno de desarrollo real para verificar su comportamiento bajo condiciones operativas y darle continuidad práctica más allá del marco del TFG. Esta proyección es, en última instancia, lo que justifica el esfuerzo invertido y lo que convierte el proyecto en una primera piedra de un recorrido profesional más amplio.

Capítulo 6. FUTURAS LÍNEAS DE TRABAJO

A lo largo del desarrollo del proyecto se han identificado varias direcciones por las que el trabajo realizado puede crecer y consolidarse en futuras iteraciones.

La primera línea de evolución consiste en sustituir el modelo de lenguaje utilizado en el laboratorio por un modelo de mayor tamaño desplegado *on-premise* sobre un hardware más potente. Esta sustitución abordaría directamente la principal limitación reconocida anteriormente, mejorando la fiabilidad del razonamiento automatizado del flujo SOAR y la consistencia del bot conversacional en tareas de *tool calling*, sin renunciar al principio de soberanía del dato sobre el que se ha diseñado toda la arquitectura.

Una segunda línea de trabajo interesante es el despliegue efectivo de un conector de *stream*. Más allá de su uso interno como mecanismo de distribución de inteligencia en tiempo real hacia sistemas consumidores, el verdadero potencial de este conector reside en habilitar la compartición de inteligencia con otras organizaciones y entidades. La construcción de comunidades de intercambio de inteligencia, especialmente entre empresas de un mismo sector o entre actores públicos y privados, constituye una de las tendencias más relevantes de la disciplina CTI y permitiría a la plataforma desplegada participar en ecosistemas de defensa colaborativa, multiplicando el valor de la inteligencia generada.

La tercera línea aborda la fragilidad estructural identificada durante el desarrollo. La incorporación de mecanismos de validación automática de esquemas en los puntos de integración con APIs de terceros, transformaría el sistema en una infraestructura robusta frente a cambios externos no anunciados. Esta línea trasciende el ámbito puramente técnico y va a juego con prácticas de ingeniería de fiabilidad propias de los servicios en producción de seguridad.

La cuarta y, posiblemente, más extensa línea de trabajo pasa por ampliar el catálogo de herramientas del agente conversacional desplegado en Discord. El patrón "agente más catálogo de herramientas" implementado está expresamente diseñado para crecer mediante la simple adición de subworkflows-herramienta, sin necesidad de modificar la lógica del agente. Las direcciones naturales de ampliación son múltiples e incluyen la consulta de IoCs en plataformas externas, el desbloqueo de IPs como operación inversa al bloqueo ya implementado, la creación automatizada de tickets en sistemas de gestión y, de forma especialmente interesante, la integración con Ansible para la ejecución de *playbooks* de respuesta sobre la infraestructura. Esta última posibilidad permite abrir el camino a una capa de respuesta activa mucho más potente que la actual, capaz de orquestar acciones complejas sobre múltiples máquinas en paralelo a partir de una única instrucción del analista en lenguaje natural.

En conjunto, estas líneas marcan un camino claro de evolución gracias al diseño modular del proyecto. Cada una puede abordarse de forma independiente sin afectar al resto del sistema, lo que asegura que el trabajo realizado en este TFG no termina con la entrega académica, sino que sirve como base para seguir creciendo.

Capítulo 7. REFERENCIAS

Las referencias bibliográficas siguen el formato APA 7ª edición, con sangría francesa y ordenación alfabética por apellido del autor o nombre de la entidad responsable.

AT & T Cybersecurity. (s.f.). *AlienVault Open Threat Exchange (OTX)*. Recuperado el 28 de abril de 2026 de <https://otx.alienvault.com/> — Fuente principal de inteligencia externa del laboratorio.

Censys. (s.f.). *Censys: The leading internet intelligence platform*. Censys, Inc. Recuperado el 28 de abril de 2026 de <https://censys.com/> — Indirectamente a través de drb-ra

Donenfeld, J. A. (2017). *WireGuard: Next generation kernel network tunnel*. En *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS)*. Internet Society. <https://www.wireguard.com/papers/wireguard.pdf> — Justifica la elección de WireGuard frente a OpenVPN (apartado 3.2.4.A).

Docker, Inc. (s.f.). *Docker Documentation*. Recuperado el 28 de abril de 2026 de <https://docs.docker.com/> — Base del despliegue contenerizado de OpenCTI, n8n y conectores propios.

drb-ra. (s.f.). *C2IntelFeeds: Automatically generated Command and Control feeds* [Repositorio de software]. GitHub. Recuperado el 28 de abril de 2026 de <https://github.com/drb-ra/C2IntelFeeds> — Fuente de los feeds C2 (Cobalt Strike, PoshC2) consumidos por el conector Multi-Feed

Filigran. (s.f.). *OpenCTI Enterprise Edition*. Recuperado el 28 de abril de 2026 de <https://filigran.io/solutions/open-cti/>

IBM. (s.f.). *¿Qué es la fatiga de alertas?* Recuperado el 28 de abril de 2026 de <https://www.ibm.com/es-es/think/topics/alert-fatigue> — Fuente conceptual del problema central del proyecto (apartado 1.3).

IBM Security. (2024). *Cost of a Data Breach Report 2024*. IBM Corporation. <https://www.ibm.com/es-es/think/insights/cost-of-a-data-breach-2024-financial-industry> — Cifras de referencia del estado del arte y del análisis de viabilidad.

IBM Security. (2025). *Cost of a Data Breach Report 2025*. IBM Corporation. <https://www.ibm.com/es-es/reports/data-breach> — Actualización utilizada para validar la estabilidad del orden de magnitud (apartado 3.5.1).

MITRE Corporation. (s.f.). *MITRE ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge*. Recuperado el 28 de abril de 2026 de <https://attack.mitre.org/> — Framework de TTPs ingerido en OpenCTI y usado por el flujo SOAR.

n8n GmbH. (s.f.). *n8n Documentation*. Recuperado el 28 de abril de 2026 de <https://docs.n8n.io/> — Documentación de la plataforma de orquestación elegida (apartado 3.2.5).

OASIS Cyber Threat Intelligence Technical Committee. (2021). *STIX™ Version 2.1: Structured Threat Information Expression* [Especificación de estándar]. OASIS Open. <https://oasis-open.github.io/cti-documentation/stix/intro.html> — Estándar sobre el que se construyen OpenCTI y los conectores propios.

OpenCTI Platform. (s.f.). *OpenCTI: Open Cyber Threat Intelligence Platform* [Repositorio de software]. GitHub. Recuperado el 28 de abril de 2026 de <https://github.com/OpenCTI-Platform/opencti> — Repositorio oficial del componente central de la arquitectura.

OpenPhish. (s.f.). *OpenPhish: Phishing Intelligence*. Recuperado el 28 de abril de 2026 de <https://openphish.com/> — Feed de URLs de phishing consumido por el conector Multi-Feed.

OPNsense Project. (s.f.). *OPNsense Documentation*. Recuperado el 28 de abril de 2026 de <https://docs.opnsense.org/> — Base de la configuración del firewall, WireGuard, API REST y Unbound DNS.

Parlamento Europeo y Consejo de la Unión Europea. (2016). Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (Reglamento General de Protección de Datos). Diario Oficial de la Unión Europea, L 119, 1-88. <https://eur-lex.europa.eu/eli/reg/2016/679/oj> Marco normativo que sustenta el principio de soberanía del dato (apartado 1.3).

Proxmox Server Solutions. (s.f.). *Proxmox VE Documentation*. Recuperado el 28 de abril de 2026 de <https://pve.proxmox.com/pve-docs/> — Documentación del hipervisor del laboratorio (apartado 3.2.1).

The Tor Project. (s.f.). Tor Bulk Exit List. Recuperado el 28 de abril de 2026 de <https://check.torproject.org/torbulkexitlist> — Lista de nodos de salida Tor consumida por el conector Multi-Feed.

U.S. Congress. (2018). *Clarifying Lawful Overseas Use of Data Act (CLOUD Act)*, H.R. 4943, 115th Congress. United States of America. <https://www.congress.gov/bill/115th-congress/house-bill/4943> — Marco extraterritorial que justifica el descarte de cloud estadounidense (apartado 1.3).

Wazuh, Inc. (s.f.). *Wazuh Documentation*. Recuperado el 28 de abril de 2026 de <https://documentation.wazuh.com/> — Base de la instalación nativa del SIEM/XDR, FIM e integración con n8n.

Capítulo 8. ANEXOS

La documentación técnica complementaria del proyecto no se reproduce íntegramente en el cuerpo de esta memoria por su extensión y naturaleza (manuales de configuración, código fuente y definiciones de workflows). Este capítulo enumera los anexos referenciados a lo largo de la memoria e indica dónde consultarlos.

- **Bloque A — Infraestructura base**
 - Anexo A.1 — Configuración VM Router
 - Anexo A.2 — OPNsense
- **Bloque B — Plataforma de ciberinteligencia**
 - Anexo B.1 — OpenCTI
 - Anexo B.2 — opencti-control
 - Anexo B.3 — Conector Multi-Feed
 - Anexo B.4 — Conector Gemini Enrichment
- **Bloque C — Detección y monitorización**
 - Anexo C.1 — Wazuh
- **Bloque D — Orquestación y automatización**
 - Anexo D.1 — n8n (instalación)
 - Anexo D.2 — Workflow SOAR
 - Anexo D.3 — Bot SOC Discord

El material se entrega en un archivo comprimido junto a esta memoria. El archivo contiene el documento del TFG y una carpeta con la documentación técnica mantenida en Obsidian durante el proyecto, donde se encuentran los manuales de cada anexo en formato Markdown junto a sus recursos gráficos.